

howzit, is there data in the tables?

not sure if im missing something 🤔

i got thta from dean and read thew gorup chat

am i shitting the bed somewhere

[11:53, 10/08/2026] Dean Feldman: has the data been linked to the frontend or should i do it?

[12:01, 10/08/2026] Nadav Sundy: how do we connect the database to test locally?

[12:02, 10/08/2026] Dean Feldman: with the endpoints?

[12:04, 10/08/2026] Nadav Sundy: What you need from your team

You now need the shared Supabase session-pooler connection string, probably in this general form:

postgresql://<user>:

<password>@<something>.pooler.supabase.com:5432/<database>

[12:04, 10/08/2026] Nadav Sundy: The frontend and backend are now running, but database-backed functionality is unavailable because the current DATABASE_URL points to local PostgreSQL while the backend expects the documented Supabase session-pooler connection.

[12:04, 10/08/2026] Nadav Sundy: this is what i get from chat

[12:08, 10/08/2026] Dean Feldman: postgresql://postgres.dbujjthhdfddghbxqjek:

[YOUR-PASSWORD]@aws-0-eu-central-1.pooler.supabase.com:5432/postgres

[12:09, 10/08/2026] Dean Feldman: password:database password that was chosen when the Supabase project was created. @Ben Swartz

[12:12, 10/08/2026] Nadav Sundy: The signin and signup buttons are there, it just needs to be linked properly to the OAuth @Gabriel Raz

Diagnosed empty database state and identified communication gaps

No, you're not. Three separate things going on, and only one of them is yours to answer.

There is no data in the tables. The database is empty. The last thing you did on #27 was a `db:migrate:down` and `db:migrate` to apply the umpire comment change, which dropped the 14 validation fixtures. The schema is there, the rows aren't. You flagged this in the PR's known limitations, but that's buried in a merged PR and nobody read it.

Nadav's question is already answered in the docs. ADR-003 consequence 3 says every developer needs the pooler connection string and the committed CA certificate, and that both are documented in `docs/development/setup.md` precisely because neither is discoverable from the Supabase dashboard. He's asking an AI instead of reading the setup guide — which either means the guide isn't discoverable, or nobody's been pointed at it. Worth knowing which.

"Has the data been linked to the frontend" is the one that worries me. There are no read endpoints. #51 is the submission endpoint and it's unstarted; nothing exposes fixtures, events or statistics over HTTP yet. The frontend can't be linked to data because the API doesn't exist. Dean seems to think it's a wiring job.

Two things to say plainly, one caution:

Loading the full corpus is **blocked**, not just slow. ADR-003 says no bulk ingestion until the storage decision is made — 3.2M deliveries against a 500 MB free plan — and that decision

has no owner and no date. Also ingestion runs at ~68 seconds per fixture, so even a few hundred matches is hours.

And the database password shouldn't go in the group chat. Dean's paste has the placeholder so nothing leaked, but the answer is "it's in `setup.md`", not "here it is in WhatsApp."

Want me to write the reply?

what must i reply to dean and then lets get started on more workd

A Full picture B Shorter

Howzit — no data in the tables at the moment. I dropped and reapplied the migration at the end of #27 to get a schema comment in, which cleared the 14 validation fixtures I'd loaded. Schema is there, rows aren't. Easy to reload a handful when we need them.

On linking to the frontend though — there aren't any read endpoints yet. #51 is the submission endpoint and it's unstarted, and nothing exposes fixtures, events or stats over HTTP. So it's not a wiring job, the API still has to be built.

Two other things worth knowing before anyone tries to bulk load:

ADR-003 says no bulk ingestion until we've decided the storage question — full corpus is 3.2M deliveries, roughly 1.3GB in Postgres, and the free plan is 500MB. That decision still has no owner and no date. Worth putting on Tuesday.

And ingestion is currently ~68 seconds per fixture (round trip per delivery), so even a few hundred matches is hours. Needs a proper batch path before we load anything real.

@Nadav — pooler string and the CA cert are both in docs/development/setup.md, that's why they're documented there. Don't paste the password in the chat.



Send whichever fits the group's tone.

Now — what next. Three candidates, and I'd argue for a specific one:

Reload validation fixtures. Twenty minutes. Unblocks anyone poking at the database and stops "is there data" recurring. Low value beyond that.

The four #27 follow-up issues. Half an hour, and the super-over exclusion one genuinely matters — if someone writes derivation before it's recorded, career figures go wrong across 99 matches and validation won't catch it because the reference scorecards won't include super-over runs either.

Ingestion performance. The real problem. 68 seconds per fixture blocks the storage measurement, blocks any realistic dataset, and blocks Sprint 2's batch pipeline. It's also

probably a cheap fix — the script does a round trip per delivery inside one transaction, and batching the inserts could plausibly be 50× faster.

My recommendation: **write the four issues first** (thirty minutes, and it stops knowledge sitting only in this conversation), then take on ingestion performance as your next piece of work. It's yours by continuity, it's the actual blocker, and fixing it makes the storage decision answerable rather than deferred.

Say go and I'll draft the four issues.

should we not work on active issue instead of writing new ones?

Fair challenge, and mostly right — but not entirely.

Where you're right: writing issues is not progress, and Sprint 1 has sixteen days. Four new issues nobody works on is worse than three you finish. If the choice is issue-writing versus building, build.

Where I'd push back: the super-over exclusion is the exception, and it's about two minutes to record. It's not a task, it's a landmine. Someone will write batting and bowling aggregates, they'll validate against published scorecards, and the scorecards exclude super-over runs too — so the bug passes validation and surfaces much later. Right now that knowledge exists in one conversation on your machine. That's worth two minutes regardless of what else you do.

The other three can wait or never be written.

So: record that one, then pick up active work. On which:

Nothing on the board is assigned to you. #46 and #63 are the ones you're closest to — you built the schema and the reference validation — and they overlap enough to do together in roughly a day. Seed data, an ERD, an invalid submission example.

Doing them also fixes the thing Dean just asked about: a repeatable seed means "is there data" gets answered by running a command rather than by asking you.

Claim #46 and #63, or is something else more urgent from the group's side?

so ill claim 46 and 63 and then what?

Claim both, then set them up properly before writing code — your Definition of Ready needs an assignee, milestone, board column and acceptance criteria on each, and #46's criteria need the edit we discussed.

Then the work itself, in this order:

Seed data first, because both issues need it and #63's other criteria sit on top of it. That means deciding what a seed actually is here: your ingestion script already loads a real Cricsheet match idempotently, so the question is whether seeding means "run the ingester over a fixed set of match files" or "a SQL file of INSERT statements". The first reuses working code and stays honest to the event model; the second is faster to run and doesn't depend on `data/` being downloaded. That's a real decision and it affects CI.

Then the invalid submission example for #63 — a match file with a deliberate defect that the ingester rejects, proving validation exists.

Then the ERD for #46, generated from the live schema rather than drawn by hand, so it can't drift.

Before any of that, one thing to check: whether a seed script already exists from someone else's work. `database/seeds/` is in the repository structure document.

```
powershell  
  
ls database/seeds -Recurse
```

Paste that, and tell me your instinct on the seed question — ingester over fixed files, or SQL inserts.

wait, again step by step, tell me what to do

Step 1 — check whether a seed already exists.

```
powershell  
  
ls database/seeds -Recurse
```

Paste the output. If someone has already put something there, that changes what we build.

```
S C:\Users\BenSwartz\Sport-Analytics-Tool> ls database/seeds -Recurse  
Directory: C:\Users\BenSwartz\Sport-Analytics-Tool\database\seeds  
Mode                LastWriteTime         Length Name  
----                -  
-a----      05/08/2026   17:13           161 README.md  
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

Empty apart from a README. Nothing to build on, nothing to conflict with.

Step 2 — read the README, since it may already state an intended approach and I'd rather follow it than contradict it:

```
powershell
```

```
Get-Content database/seeds/README.md
```

Paste it.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> Get-Content
database/seeds/README.md
# Seeds
Store deterministic, non-sensitive development and test seed data here. Seed data
must be clearly fictional or sourced under an appropriate licence.
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

"Deterministic" and "sourced under an appropriate licence" — Cricsheet data qualifies on the licence (it's CC BY 4.0, though verify that yourself rather than taking my word), and real match files are deterministic in a way generated fixtures aren't.

But there's a decision to make first, and it shapes everything after.

Step 3 — decide the seed mechanism. Two options:

A. Seed = run the ingester over a fixed set of match files. Reuses code that already works and is proven idempotent. Stays honest to the event model: the seed is real event data, not hand-written totals. But it depends on `data/` being downloaded, which is 456 MB and gitignored, so a fresh clone can't seed without running Nadav's downloader first. And it's slow — 68 seconds per fixture.

B. Seed = SQL files of INSERT statements, committed to the repository. Fast, self-contained, works in CI on a fresh clone with no download. But it's derived output committed as source, so it goes stale when the schema changes, and someone has to regenerate it.

C. A hybrid — commit the four or five match JSON files the seed needs directly under `database/seeds/`, and seed by running the ingester over those. Self-contained, no download needed, real event data, and the schema can't drift from it because the ingester is the same code path. Slow-ish, but four fixtures is four minutes.

I'd take C. It satisfies "deterministic", it doesn't commit derived output, and it means the seed exercises the ingestion path rather than bypassing it — which matters because #63 also wants an invalid submission example, and that only works if the seed goes through validation.

Your call. Which one?

you decide

C — commit the match files, seed by running the ingester over them.

Step 4 — create the seed data directory and copy the fixtures in.

The four you validated are the right set: 729307 has the published scorecard evidence, 423788 covers the super over, 1462921 and 1399114 cover both miscounted-over source types.

powershell

```
New-Item -ItemType Directory -Force -Path database\seeds\matches | Out-Null
Copy-Item data\cricsheet\matches\indian-premier-league\729307.json database\seeds\ma
Copy-Item data\cricsheet\matches\australia-in-new-zealand-t20i-series\423788.json data
Copy-Item data\cricsheet\matches\afrika-continental-cup\1462921.json database\seeds\ma
Copy-Item data\cricsheet\matches\asian-games-men-s-cricket-competition\1399114.json da
ls database\seeds\matches | Select-Object Name, Length
```

Paste the listing — I want the file sizes, since four match files committed to the repository is a size question worth being deliberate about.

Before the next step, one thing to check that I can't: **the Cricsheet licence**. I believe it's CC BY 4.0, but the seeds README requires an appropriate licence and I'd rather you confirm from the source than take my recollection. Nadav's [docs/data/cricsheet.md](#) may already state it — check there first.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> Get-Content
database/seeds/README.md
# Seeds
Store deterministic, non-sensitive development and test seed data here. Seed data
must be clearly fictional or sourced under an appropriate licence.
PS C:\Users\BenSwartz\Sport-Analytics-Tool> New-Item -ItemType Directory -Force
-Path database\seeds\matches | Out-Null
>> Copy-Item data\cricsheet\matches\indian-premier-league\729307.json
database\seeds\matches\
>> Copy-Item data\cricsheet\matches\australia-in-new-zealand-t20i-
series\423788.json database\seeds\matches\
>> Copy-Item data\cricsheet\matches\afrika-continental-cup\1462921.json
database\seeds\matches\
>> Copy-Item data\cricsheet\matches\asian-games-men-s-cricket-
competition\1399114.json database\seeds\matches\
>> ls database\seeds\matches | Select-Object Name, Length
Name      Length
----      -
1399114.json 89267
1462921.json 79605
423788.json  92217
729307.json  84861
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

346 KB total. Fine to commit.

Step 5 — check the licence. The seeds README requires it and I'm not going to guess:

powershell

```
Select-String -Path docs\data\cricsheet.md -Pattern "licence|license|CC BY|attribution"
```

Paste the output. If Nadav documented it, we cite his page. If he didn't, that's a gap in #28 worth flagging, and you'd need to check cricsheet.org yourself before committing data files to the repository.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> Select-String -Path  
docs\data\cricsheet.md -Pattern "licence|license|CC BY|attribution|terms" -  
Context 2,2  
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

Nothing. The data-source documentation says nothing about licensing, which is a real gap — you have 456 MB of third-party data in the project and the page describing where it came from doesn't record the terms it came under.

That matters more now than it did an hour ago, because committing four of those files into the repository redistributes them, which is a stronger act than downloading them for local use.

Let me check the actual terms rather than relying on memory.

Searched the web

My recollection was wrong — it's **not** CC BY 4.0.

The Cricsheet Register is made available under the Open Data Commons Attribution License, which permits sharing, creating works from the dataset, and adapting it, provided any public use is attributed in the manner the licence specifies. That's the Register page, though, not the match data downloads. Fetch the downloads page directly rather than inferring:

Fetches: Available match data downloads - Cricsheet

The downloads page doesn't state the licence — it's on the Register page and presumably the About page. What the Register page says is that data is made available under the **Open Data Commons Attribution License** (ODC-BY 1.0), with the human-readable summary permitting copying, distributing and using the dataset, producing works from it, and adapting it, provided public use is attributed as the licence specifies.

That covers your case: committing four match files and publishing derived statistics is permitted, with attribution.

Two things follow, and one is more urgent than the seed work:

Check the withheld matches. The downloads page notes 370 matches are withheld, involving Afghanistan or the Afghanistan Premier League, for reasons Cricsheet explains in a linked article. Nadav's corpus was downloaded before now, and if the counts in your documentation predate that withholding — or the corpus contains matches Cricsheet has since pulled — that's worth knowing. Not a licence problem, but a data-provenance one.

The licence gap is Nadav's `docs/data/cricsheet.md`. He documented the downloader and the scope but not the terms, and 456 MB of third-party data with no recorded licence is a real omission for a project publishing datasets. Worth a comment on #28.

Step 6 — record the licence where your seed lives. Update `database/seeds/README.md`:

```
powershell
```

```
@'
```

```
# Seeds
```

Store deterministic, non-sensitive development and test seed data here. Seed data must be clearly fictional or sourced under an appropriate licence.

```
## Match data
```

`matches/` contains four Cricsheet match files used to seed a development database. They are loaded by the ingestion script rather than by SQL inserts, so the seed exercises the same validation path as a real submission and cannot drift from the schema.

```
| Match | Reason for inclusion |
|---|---|
| 729307 | Reference fixture. Expected figures recorded in `evidence/validation/729307
| 423788 | Super over. Four innings, reversed batting order, tie decided by eliminator
| 1462921 | Miscalculated over with the ball count supplied as a string. |
| 1399114 | Miscalculated over with the ball count supplied as an integer. |
```

```
## Licence and attribution
```

Cricsheet match data is made available under the Open Data Commons Attribution License 1.0 (ODC-BY), which permits copying, distributing, using, adapting and producing works from the dataset provided public use is attributed.

Source: Cricsheet, <https://cricsheet.org/>

Any public release of data derived from these files must carry the same attribution.

```
## AI Declaration
```

The preceding document was generated with the assistance of Claude-Web[Claude Opus 5].

```
'@ | Set-Content -Path database\seeds\README.md -Encoding utf8
```

Run that, then we write the seed command itself.

Step 7 — write the seed script. Create `apps/backend/scripts/seed.ts`:

ts

```

/**
 * Load the development seed fixtures into the database.
 *
 * The seed runs the ingestion script over the match files committed under
 * database/seeds/matches, so that seeding exercises the same validation and
 * insertion path as a real submission and cannot drift from the schema.
 *
 * Ingestion is idempotent, so running the seed twice leaves the database in the
 * same state.
 *
 * Usage:
 *   npm run db:seed --workspace=@sport-analytics/backend
 */

import { readdirSync } from 'node:fs';
import { resolve } from 'node:path';
import { execFileSync } from 'node:child_process';

const seedDir = resolve(__dirname, '../../../database/seeds/matches');

const files = readdirSync(seedDir)
  .filter((name) => name.endsWith('.json'))
  .sort();

if (files.length === 0) {
  console.error(`No seed match files found in ${seedDir}`);
  process.exit(1);
}

console.log(`Seeding ${files.length} fixtures from ${seedDir}`);

for (const file of files) {
  console.log(`\n--- ${file} ---`);
  execFileSync(
    process.execPath,
    [
      resolve(__dirname, '../../../node_modules/tsx/dist/cli.mjs'),
      '--env-file=.env',
      resolve(__dirname, 'ingest-match.ts'),
      resolve(seedDir, file),
    ],
    { stdio: 'inherit', cwd: resolve(__dirname, '..') }
  );
}

console.log('\nSeed complete.');
```

Add to `apps/backend/package.json` scripts:

```
json
```

```
"db:seed": "tsx --env-file=.env scripts/seed.ts"
```

The tsx path there is a guess — I don't know how your install resolves it. Check before running:

```
powershell
```

```
ls node_modules\tsx\dist\cli.mjs
```

If that doesn't exist, paste what `ls node_modules\tsx\dist` gives you and I'll adjust. There's a simpler approach using `npm run db:ingest` per file, but it spawns a shell per match and is slower; I'd rather get this one right.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> ls node_modules\tsx\dist\cli.mjs
Directory: C:\Users\BenSwartz\Sport-Analytics-Tool\node_modules\tsx\dist
Mode                LastWriteTime         Length Name
----                -
-a----      05/08/2026   17:04         121766 cli.mjs
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

Path is right. Create the file and add the script line, then run it:

```
powershell
```

```
npm run db:seed --workspace=@sport-analytics/backend
```

Paste the output. Expect four ingest summaries — 423788 at 261, 729307 at 237, 1399114 at 241, 1462921 at 216, in filename order — and roughly four minutes given the ingestion rate.

The database is currently empty, so all four should report deliveries added rather than no-ops. Run it a second time afterwards and every one should report zero; that's the repeatability criterion #46 and #63 both ask for, and it's worth having the evidence.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> npm run db:seed --
workspace=@sport-analytics/backend
> @sport-analytics/backend@0.1.0 db:seed
> tsx --env-file=.env scripts/seed.ts
Seeding 4 fixtures from C:\Users\BenSwartz\Sport-Analytics-
Tool\database\seeds\matches
--- 1399114.json ---
Ingested 1399114
  fixture_id    : 1
  people resolved : 28
  deliveries added : 241
```

```
--- 1462921.json ---
Ingested 1462921
  fixture_id    : 2
  people resolved : 26
  deliveries added : 216
--- 423788.json ---
Ingested 423788
  fixture_id    : 3
  people resolved : 27
  deliveries added : 261
--- 729307.json ---
Ingested 729307
  fixture_id    : 4
  people resolved : 27
  deliveries added : 237
Seed complete.
PS C:\Users\BenSwartz\Sport-Analytics-Tool>

Show more
```

All four loaded, 955 deliveries, fixture ids 1-4 with no gaps.

Step 8 — prove repeatability. Run it again:

```
powershell
npm run db:seed --workspace=@sport-analytics/backend
```

Every fixture should report zero deliveries added and the no-op message. That's the evidence for "seed loading is repeatable" on both #46 and #63 — and it's stronger than a typical seed script, because idempotency here comes from the natural key rather than from a truncate-and-reload.

Paste the output.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> npm run db:seed --
workspace=@sport-analytics/backend
> @sport-analytics/backend@0.1.0 db:seed
> tsx --env-file=.env scripts/seed.ts
Seeding 4 fixtures from C:\Users\BenSwartz\Sport-Analytics-
Tool\database\seeds\matches
--- 1399114.json ---
Ingested 1399114
  fixture_id    : 1
  people resolved : 28
  deliveries added : 0
  (already present: this run was a no-op, which is the intended behaviour)
--- 1462921.json ---
Ingested 1462921
  fixture_id    : 2
  people resolved : 26
```

```
deliveries added : 0
(already present: this run was a no-op, which is the intended behaviour)
--- 423788.json ---
Ingested 423788
  fixture_id    : 3
  people resolved : 27
  deliveries added : 0
(already present: this run was a no-op, which is the intended behaviour)
--- 729307.json ---
Ingested 729307
  fixture_id    : 4
  people resolved : 27
  deliveries added : 0
(already present: this run was a no-op, which is the intended behaviour)
Seed complete.
PS C:\Users\BenSwartz\Sport-Analytics-Tool>

Show more
```

Repeatable, and the fixture ids held steady — no rows added, no sequence churn on the fixtures themselves.

That's #46's seed criterion and #63's repeatability criterion both satisfied. Commit before moving on, since the next piece is the invalid submission example and that's a separate concern.

Step 9 — branch and commit.

You'll need an issue number. #46 and #63 are both open and this work serves both. Commit against #46 since the seed lives under `database/seeds` and that's #46's territory, and reference #63 in the body.

First check you're on `main` and up to date:

```
powershell

git switch main
git pull origin main
git switch -c feat/46-add-development-seed-data
```

Then:

```
powershell
```

```
git add database/seeds apps/backend/scripts/seed.ts apps/backend/package.json
git commit -m "feat(db): add repeatable development seed data" -m "Four Cricsheet match
```

The fixtures are chosen to cover the reference scorecard, a super over, and both source

Ingestion is idempotent, so a second run adds no rows. Verified: first run loads 955 d

The Cricsheet licence and required attribution are recorded in database/seeds/README

Refs #46

Refs #63

Assisted-by: Claude-Web[Claude Opus 5]"

powershell

```
git push -u origin feat/46-add-development-seed-data
```

Paste the output.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> git switch main
>> git pull origin main
>> git switch -c feat/46-add-development-seed-data
M    apps/backend/package.json
M    database/seeds/README.md
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
remote: Failed to authenticate user
fatal: Authentication failed for 'https://sdp.ms.wits.ac.za/git-push-pray/Sport-
Analytics-Tool.git/'
Switched to a new branch 'feat/46-add-development-seed-data'
PS C:\Users\BenSwartz\Sport-Analytics-Tool> git add database/seeds
apps/backend/scripts/seed.ts apps/backend/package.json
>> git commit -m "feat(db): add repeatable development seed data" -m "Four
Cricsheet match files are committed under database/seeds/matches and loaded
by the ingestion script rather than by SQL inserts, so the seed exercises the same
validation path as a real submission and cannot drift from the schema.
>>
>> The fixtures are chosen to cover the reference scorecard, a super over, and
both source representations of a miscounted over ball count.
>>
>> Ingestion is idempotent, so a second run adds no rows. Verified: first run loads
955 deliveries across four fixtures, second run adds zero.
>>
>> The Cricsheet licence and required attribution are recorded in
database/seeds/README.md.
>>
>> Refs #46
>> Refs #63
>> Assisted-by: Claude-Web[Claude Opus 5]"
```

```
warning: in the working copy of 'database/seeds/README.md', CRLF will be
replaced by LF the next time Git touches it
warning: in the working copy of 'apps/backend/scripts/seed.ts', CRLF will be
replaced by LF the next time Git touches it
[feat/46-add-development-seed-data 5576278] feat(db): add repeatable
development seed data
7 files changed, 12777 insertions(+), 1 deletion(-)
create mode 100644 apps/backend/scripts/seed.ts
create mode 100644 database/seeds/matches/1399114.json
create mode 100644 database/seeds/matches/1462921.json
create mode 100644 database/seeds/matches/423788.json
create mode 100644 database/seeds/matches/729307.json
PS C:\Users\BenSwartz\Sport-Analytics-Tool> git push -u origin feat/46-add-
development-seed-data
Enumerating objects: 23, done.
Counting objects: 100% (23/23), done.
Delta compression using up to 12 threads
Compressing objects: 100% (15/15), done.
Writing objects: 100% (15/15), 17.01 KiB | 1.89 MiB/s, done.
Total 15 (delta 8), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a new pull request for 'feat/46-add-development-seed-data':
remote: https://sdp.ms.wits.ac.za/git-push-pray/Sport-Analytics-
Tool/pulls/new/feat/46-add-development-seed-data
remote:
remote: . Processing 1 references
remote: Processed 1 references in total
To https://sdp.ms.wits.ac.za/git-push-pray/Sport-Analytics-Tool.git
 * [new branch] feat/46-add-development-seed-data -> feat/46-add-
development-seed-data
branch 'feat/46-add-development-seed-data' set up to track 'origin/feat/46-add-
development-seed-data'.
PS C:\Users\BenSwartz\Sport-Analytics-Tool>

Show more
```

Committed and pushed. One thing to check: the `git pull` failed with the intermittent auth error, so your branch is based on whatever `main` was at your last successful pull, not necessarily the latest. Gitea will tell you if it's behind when you open the PR.

Also note `7 files changed` — the four match files, the seed script, the README and `package.json`. That's what you expect.

Step 10 — the invalid submission example, which is #63's remaining criterion and the one that proves validation exists rather than just asserting it.

Before writing it, a question worth settling: what makes a submission invalid in a way the current ingester actually catches? Looking at `ingest-match.ts`, the failures it would genuinely produce are a name absent from the registry (it throws explicitly), a delivery whose `runs.total` doesn't equal batter plus extras (`delivery_runs_ck`), or a striker equal to the non-striker (`delivery_striker_ck`).

The registry one is the most honest example — it's a real class of defect, the error message is explicit, and it demonstrates the O2 name-identity rule the schema is built around.

Which would you rather demonstrate? Or all three, as separate files?

you choose

All three, as separate files. One example proves the ingester rejects something; three prove rejection happens at different layers — application check, database check constraint, and a second check constraint — which is a stronger claim and costs almost nothing extra.

Step 11 — build the invalid files from a real match. They should be a valid match with one deliberate defect, so the defect is unambiguous.

Create `scripts/make_invalid_seeds.py`:

```
python
```

```

"""Produce invalid submission examples by introducing one defect into a valid match."""
import json
from pathlib import Path

SOURCE = Path("database/seeds/matches/729307.json")
OUT = Path("database/seeds/invalid")
OUT.mkdir(parents=True, exist_ok=True)

def load():
    with SOURCE.open(encoding="utf-8") as f:
        return json.load(f)

def write(name, data, note):
    path = OUT / name
    with path.open("w", encoding="utf-8") as f:
        json.dump(data, f, indent=2)
    print(f"{name}: {note}")

# 1. A name that is absent from the registry.
d = load()
first = d["innings"][0]["overs"][0]["deliveries"][0]
first["batter"] = "AN Unregistered"
write("unregistered-name.json", d,
      "batter renamed to a person absent from info.registry.people")

# 2. runs.total inconsistent with its parts.
d = load()
first = d["innings"][0]["overs"][0]["deliveries"][0]
first["runs"] = {"batter": 4, "extras": 0, "total": 6}
write("inconsistent-runs.json", d,
      "runs.total set to 6 where batter + extras is 4")

# 3. Striker and non-striker are the same person.
d = load()
first = d["innings"][0]["overs"][0]["deliveries"][0]
first["non_striker"] = first["batter"]
write("striker-equals-non-striker.json", d,
      "non_striker set equal to batter")

```

Run it:

```

powershell
python scripts/make_invalid_seeds.py

```

Paste the output. Then we run each through the ingester and record what it says — the point being that a rejection must explain what was wrong, which the project brief requires

explicitly.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> python
scripts/make_invalid_seeds.py
unregistered-name.json: batter renamed to a person absent from
info.registry.people
inconsistent-runs.json: runs.total set to 6 where batter + extras is
4
striker-equals-non-striker.json: non_striker set equal to batter
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

Step 12 — run each through the ingester and capture what it says.

powershell

```
npm run db:ingest --workspace=@sport-analytics/backend -- "C:\Users\BenSwartz\Sport-An
```

powershell

```
npm run db:ingest --workspace=@sport-analytics/backend -- "C:\Users\BenSwartz\Sport-An
```

powershell

```
npm run db:ingest --workspace=@sport-analytics/backend -- "C:\Users\BenSwartz\Sport-An
```

Paste all three outputs.

Two things I'm watching for. First, that each fails with a message identifying the actual problem rather than something opaque — the brief requires a rejection to tell the submitter what was wrong. Second, and more important: **the fixture already exists in the database from the seed**, so `INSERT INTO fixture ... ON CONFLICT DO NOTHING` will succeed silently and the deliveries will hit the natural-key conflict before ever reaching the defect. Two of these three may report a no-op instead of an error, which would mean the test doesn't test what it claims.

If that happens it's a useful finding, not a failure — it means the invalid examples need distinct `source_ref` values, and I'll adjust the generator.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> npm run db:ingest --
workspace=@sport-analytics/backend -- "C:\Users\BenSwartz\Sport-Analytics-
Tool\database\seeds\invalid\unregistered-name.json"
> @sport-analytics/backend@0.1.0 db:ingest
> tsx --env-file=.env scripts/ingest-match.ts C:\Users\BenSwartz\Sport-Analytics-
```

```

Tool\database\seeds\invalid\unregistered-name.json
Ingestion failed.
null value in column "striker_id" of relation "delivery" violates not-null constraint
npm error Lifecycle script db:ingest failed with error:
npm error code 1
npm error path C:\Users\BenSwartz\Sport-Analytics-Tool\apps\backend
npm error workspace @sport-analytics/backend@0.1.0
npm error location C:\Users\BenSwartz\Sport-Analytics-Tool\apps\backend
npm error command failed
npm error command C:\windows\system32\cmd.exe /d /s /c tsx --env-file=.env
scripts/ingest-match.ts C:\Users\BenSwartz\Sport-Analytics-
Tool\database\seeds\invalid\unregistered-name.json
PS C:\Users\BenSwartz\Sport-Analytics-Tool> npm run db:ingest --
workspace=@sport-analytics/backend -- "C:\Users\BenSwartz\Sport-Analytics-
Tool\database\seeds\invalid\inconsistent-runs.json"
> @sport-analytics/backend@0.1.0 db:ingest
> tsx --env-file=.env scripts/ingest-match.ts C:\Users\BenSwartz\Sport-Analytics-
Tool\database\seeds\invalid\inconsistent-runs.json
Ingestion failed.
new row for relation "delivery" violates check constraint "delivery_runs_ck"
npm error Lifecycle script db:ingest failed with error:
npm error code 1
npm error path C:\Users\BenSwartz\Sport-Analytics-Tool\apps\backend
npm error workspace @sport-analytics/backend@0.1.0
npm error location C:\Users\BenSwartz\Sport-Analytics-Tool\apps\backend
npm error command failed
npm error command C:\windows\system32\cmd.exe /d /s /c tsx --env-file=.env
scripts/ingest-match.ts C:\Users\BenSwartz\Sport-Analytics-
Tool\database\seeds\invalid\inconsistent-runs.json
PS C:\Users\BenSwartz\Sport-Analytics-Tool> npm run db:ingest --
workspace=@sport-analytics/backend -- "C:\Users\BenSwartz\Sport-Analytics-
Tool\database\seeds\invalid\striker-equals-non-striker.json"
> @sport-analytics/backend@0.1.0 db:ingest
> tsx --env-file=.env scripts/ingest-match.ts C:\Users\BenSwartz\Sport-Analytics-
Tool\database\seeds\invalid\striker-equals-non-striker.json
Ingestion failed.
new row for relation "delivery" violates check constraint "delivery_striker_ck"
npm error Lifecycle script db:ingest failed with error:
npm error code 1
npm error path C:\Users\BenSwartz\Sport-Analytics-Tool\apps\backend
npm error workspace @sport-analytics/backend@0.1.0
npm error location C:\Users\BenSwartz\Sport-Analytics-Tool\apps\backend
npm error command failed
npm error command C:\windows\system32\cmd.exe /d /s /c tsx --env-file=.env
scripts/ingest-match.ts C:\Users\BenSwartz\Sport-Analytics-
Tool\database\seeds\invalid\striker-equals-non-striker.json
PS C:\Users\BenSwartz\Sport-Analytics-Tool>

```

Show more

All three rejected. Two of them cleanly — `delivery_runs_ck` and `delivery_striker_ck` both fired, and the constraint names tell you exactly which rule was broken.

The first one is a defect, and a real one. The registry check in `ingest-match.ts` is supposed to throw `Name "AN Unregistered" is absent from this match's registry` — an explicit, readable error. It never ran. Instead the insert went ahead with a null `striker_id` and the database caught it with a not-null violation, which tells the submitter nothing about the actual problem.

Why: `sourceRefFor()` is defined but never called. The insert uses `personId.get(delivery.batter)`, and a `Map.get` miss returns `undefined`, which `pg` sends as null. So the guard function exists and is dead code.

That matters beyond this test. The project brief requires a rejection to tell the submitter what was wrong rather than failing quietly, and "null value in column striker_id" is exactly the failure mode it's warning against. It's also the O2 name-identity rule going unenforced at the point it matters most.

Fix in `ingest-match.ts`. Find the delivery insert's parameter array:

```
ts

    personId.get(delivery.batter),
    personId.get(delivery.non_striker),
    personId.get(delivery.bowler),
```

Replace with:

```
ts

    requirePerson(delivery.batter, 'batter'),
    requirePerson(delivery.non_striker, 'non-striker'),
    requirePerson(delivery.bowler, 'bowler'),
```

And add this next to `sourceRefFor`:

```
ts

/** Resolve a name to a person, refusing to proceed on a name the registry omits. */
function requirePerson(name: string, role: string): number {
  const id = personId.get(name);
  if (id === undefined) {
    throw new Error(
      `The ${role} "${name}" is absent from this match's registry. ` +
      `Names are not identifiers: every person must appear in info.registry.people.`
    );
  }
  return id;
}
```

`personId` is declared inside `main()`, so `requirePerson` has to be declared inside `main()` too — put it just after the `const personId = new Map...` line rather than at module scope.

Make the change, then re-run the first file. Paste what it says.

```
/** * Load a single Cricshet match into the delivery event schema. * * This script exists to validate the schema against a published scorecard. It is * not t...
```

PASTED

can you please give me the whole file

Restructured code integration and resolved formatting inconsistencies

Full file. Two changes from yours: `requirePerson` added inside `main()` after `personId` is populated, and `sourceRefFor` removed — it was dead code and `requirePerson` supersedes it. I've also fixed the indentation on `let legalBalls`.

ts

```

/**
 * Load a single Cricsheet match into the delivery event schema.
 *
 * This script exists to validate the schema against a published scorecard. It is
 * not the ingestion pipeline, which is a separate concern: it loads one match,
 * makes no attempt to be fast, and holds everything in memory.
 *
 * It is idempotent. Running it twice produces the same database state, which is
 * the property required of a replayable feed.
 *
 * Usage:
 *   npm run db:ingest --workspace=@sport-analytics/backend -- <path-to-match.json>
 */

```

```

import { readFileSync } from 'node:fs';
import { createHash } from 'node:crypto';
import { basename } from 'node:path';
import { Client } from 'pg';

interface Delivery {
  actual_delivery?: string;
  batter: string;
  bowler: string;
  non_striker: string;
  runs: { batter: number; extras: number; total: number; non_boundary?: boolean };
  extras?: Record<string, number>;
  wickets?: Array<{
    kind: string;
    player_out: string;
    fielders?: Array<{ name?: string; substitute?: boolean }>;
  }>;
  review?: {
    by: string;
    umpire?: string;
    batter?: string;
    decision: string;
    type?: string;
  };
  replacements?: {
    role?: Array<{ in: string; out?: string; reason?: string; role?: string }>;
    player?: Array<{ in: string; out?: string; reason?: string }>;
  };
}

const matchPath = process.argv[2];
if (!matchPath) {
  console.error('Usage: db:ingest -- <path-to-match.json>');
  process.exit(1);
}

```

```

}

const raw = readFileSync(matchPath);
const match = JSON.parse(raw.toString('utf8'));
const info = match.info;
const meta = match.meta ?? {};
const registry: Record<string, string> = info.registry?.people ?? {};

const client = new Client({
  connectionString: process.env.DATABASE_URL,
  ssl: {
    rejectUnauthorized: true,
    ca: readFileSync(new URL('../certs/supabase-ca.crt', import.meta.url), 'utf8'),
  },
});

async function scalar<T>(sql: string, values: unknown[] = []): Promise<T> {
  const { rows } = await client.query(sql, values);
  return rows[0][Object.keys(rows[0])[0]] as T;
}

async function main(): Promise<void> {
  await client.connect();
  await client.query('BEGIN');

  const sourceRef = basename(matchPath).replace(/\.json$/, '');

  // ---- submission -----
  const submissionId = await scalar<number>(
    `INSERT INTO submission (source_filename, source_sha256, status)
    VALUES ($1, $2, 'accepted')
    RETURNING submission_id`,
    [basename(matchPath), createHash('sha256').update(raw).digest('hex')]
  );

  // ---- people -----
  const personId = new Map<string, number>();
  for (const [name, ref] of Object.entries(registry)) {
    const id = await scalar<number>(
      `INSERT INTO person (source_ref, display_name) VALUES ($1, $2)
      ON CONFLICT (source_ref) DO UPDATE SET display_name = EXCLUDED.display_name
      RETURNING person_id`,
      [ref, name]
    );
    await client.query(
      `INSERT INTO person_alias (person_id, name, first_seen) VALUES ($1, $2, $3)
      ON CONFLICT DO NOTHING`,
      [id, name, info.dates[0]]
    );
  }
}

```

```

    personId.set(name, id);
}

/**
 * Resolve a name to a person, refusing to proceed on a name the registry
 * omits. Names are not identifiers: a name absent from info.registry.people
 * cannot be resolved to anyone, and inserting a null in its place would
 * produce a constraint error that says nothing about the real problem.
 */
function requirePerson(name: string, role: string): number {
    const id = personId.get(name);
    if (id === undefined) {
        throw new Error(
            `The ${role} "${name}" is absent from this match's registry. ` +
            `Every person referenced by a delivery must appear in info.registry.people.`
        );
    }
    return id;
}

// ---- teams, venue, competition -----
const teamId = new Map<string, number>();
for (const name of info.teams as string[]) {
    teamId.set(
        name,
        await scalar<number>(
            `INSERT INTO team (name) VALUES ($1)
            ON CONFLICT (name) DO UPDATE SET name = EXCLUDED.name
            RETURNING team_id`,
            [name]
        )
    );
}

const venueId = await scalar<number>(
    `INSERT INTO venue (name, city) VALUES ($1, $2)
    ON CONFLICT ON CONSTRAINT venue_name_city_key DO UPDATE SET name = EXCLUDED.name
    RETURNING venue_id`,
    [info.venue, info.city ?? null]
);

let competitionId: number | null = null;
if (info.event?.name) {
    competitionId = await scalar<number>(
        `INSERT INTO competition (name) VALUES ($1)
        ON CONFLICT (name) DO UPDATE SET name = EXCLUDED.name
        RETURNING competition_id`,
        [info.event.name]
    );
}

```

```

}

// ---- fixture -----
const outcome = info.outcome ?? {};
let outcomeKind = 'no result';
if (outcome.winner) outcomeKind = 'won';
else if (outcome.result === 'tie') outcomeKind = 'tie';
else if (outcome.result === 'draw') outcomeKind = 'draw';

const dates = info.dates as string[];

await client.query(
  `INSERT INTO fixture (
    source_ref, competition_id, event_match_number, event_group, event_stage,
    season, match_type, team_type, gender, balls_per_over, scheduled_overs,
    venue_id, start_date, end_date, toss_winner_id, toss_decision,
    toss_uncontested, outcome, winner_id, eliminator_id, outcome_by_runs,
    outcome_by_wickets, outcome_method, decided_by_bowl_out, missing_fields,
    source_version, source_revision, first_seen_in)
  VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11,$12,$13,$14,$15,$16,$17,$18,$19,
    $20,$21,$22,$23,$24,$25,$26,$27,$28)
  ON CONFLICT (source_ref) DO NOTHING`,
  [
    sourceRef,
    competitionId,
    info.event?.match_number ?? null,
    info.event?.group ?? null,
    info.event?.stage ?? null,
    String(info.season),
    info.match_type,
    info.team_type,
    info.gender,
    info.balls_per_over,
    info.overs ?? null,
    venueId,
    dates[0],
    dates[dates.length - 1],
    teamId.get(info.toss.winner) ?? null,
    info.toss.decision,
    Boolean(info.toss.uncontested),
    outcomeKind,
    outcome.winner ? teamId.get(outcome.winner) : null,
    outcome.eliminator ? (teamId.get(outcome.eliminator) ?? null) : null,
    outcome.by?.runs ?? null,
    outcome.by?.wickets ?? null,
    outcome.method ?? null,
    Boolean(outcome.bowl_out),
    info.missing ?? [],
    meta.data_version ?? 'unknown',
  ]
);

```

```

    meta.revision ?? 0,
    submissionId,
  ]
);

const fixtureId = await scalar<number>(
  'SELECT fixture_id FROM fixture WHERE source_ref = $1',
  [sourceRef]
);

for (const [ordinal, name] of (info.teams as string[]).entries()) {
  await client.query(
    `INSERT INTO fixture_team (fixture_id, team_id, ordinal) VALUES ($1,$2,$3)
    ON CONFLICT DO NOTHING`,
    [fixtureId, teamId.get(name), ordinal + 1]
  );
}

for (const [team, players] of Object.entries(info.players ?? {})) {
  for (const name of players as string[]) {
    await client.query(
      `INSERT INTO fixture_squad (fixture_id, person_id, team_id, role)
      VALUES ($1,$2,$3,$4) ON CONFLICT DO NOTHING`,
      [
        fixtureId,
        personId.get(name),
        teamId.get(team),
        info.supersubs?.[team] === name ? 'supersub' : null,
      ]
    );
  }
}

for (const [role, names] of Object.entries(info.officials ?? {})) {
  for (const name of names as string[]) {
    const officialId = await scalar<number>(
      `INSERT INTO official (display_name) VALUES ($1)
      ON CONFLICT DO NOTHING RETURNING official_id`,
      [name]
    ).catch(async () =>
      scalar<number>('SELECT official_id FROM official WHERE display_name = $1', [name]
    );
    await client.query(
      `INSERT INTO fixture_official (fixture_id, official_id, role)
      VALUES ($1,$2,$3) ON CONFLICT DO NOTHING`,
      [fixtureId, officialId, role]
    );
  }
}

```

```

for (const name of (info.player_of_match ?? []) as string[]) {
  await client.query(
    `INSERT INTO fixture_player_of_match (fixture_id, person_id) VALUES ($1,$2)
    ON CONFLICT DO NOTHING`,
    [fixtureId, personId.get(name)]
  );
}

// ---- innings and deliveries -----
let deliveryCount = 0;

for (const [ordinal, innings] of (match.innings as any[]).entries()) {
  await client.query(
    `INSERT INTO innings (fixture_id, ordinal, batting_team_id, is_super_over,
      declared, forfeited, target_runs, target_overs,
      penalty_pre, penalty_post)
    VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10)
    ON CONFLICT (fixture_id, ordinal) DO NOTHING`,
    [
      fixtureId,
      ordinal,
      teamId.get(innings.team),
      Boolean(innings.super_over),
      Boolean(innings.declared),
      Boolean(innings.forfeited),
      innings.target?.runs ?? null,
      innings.target?.overs ?? null,
      innings.penalty_runs?.pre ?? null,
      innings.penalty_runs?.post ?? null,
    ]
  );

  const inningsId = await scalar<number>(
    `SELECT innings_id FROM innings WHERE fixture_id = $1 AND ordinal = $2`,
    [fixtureId, ordinal]
  );

  for (const powerplay of innings.powerplays ?? []) {
    await client.query(
      `INSERT INTO innings_powerplay (innings_id, from_ball, to_ball, type)
      VALUES ($1,$2,$3,$4) ON CONFLICT DO NOTHING`,
      [inningsId, powerplay.from, powerplay.to, powerplay.type]
    );
  }

  for (const name of innings.absent_hurt ?? []) {
    await client.query(
      `INSERT INTO innings_absent (innings_id, person_id) VALUES ($1,$2)

```

```

    ON CONFLICT DO NOTHING`,
    [inningsId, personId.get(name)]
  );
}

// The source supplies the ball count as a string in some matches and an
// integer in others; coerce it.
for (const [over, detail] of Object.entries(innings.miscounted_overs ?? {})) {
  await client.query(
    `INSERT INTO innings_miscounted_over (innings_id, over_number, balls)
    VALUES ($1,$2,$3) ON CONFLICT DO NOTHING`,
    [inningsId, Number(over), Number((detail as { balls: string | number }).balls)
  );
}

let sequence = 0;

for (const over of innings.overs ?? []) {
  let legalBalls = 0;

  for (const [position, delivery] of (over.deliveries as Delivery[]).entries()) {
    sequence += 1;
    const extras = delivery.extras ?? {};

    // The printed ball number counts legal deliveries only. Wides and
    // no-balls do not advance it, so it repeats within an over. It is a
    // label, never an identifier.
    const isLegal = extras.wides === undefined && extras.noballs === undefined;
    if (isLegal) legalBalls += 1;
    const ballNumber = `${over.over}.${Math.max(legalBalls, 1)}`;

    const { rows } = await client.query(
      `INSERT INTO delivery (
        innings_id, over_number, position_in_over, innings_sequence,
        ball_number, striker_id, non_striker_id, bowler_id,
        runs_off_bat, runs_extras, runs_total, non_boundary,
        extra_wides, extra_noballs, extra_byes, extra_legbyes, extra_penalty,
        submission_id)
      VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11,$12,$13,$14,$15,$16,$17,$18)
      ON CONFLICT (innings_id, over_number, position_in_over)
      WHERE superseded_at IS NULL
      DO NOTHING
      RETURNING delivery_id`,
      [
        inningsId,
        over.over,
        position,
        sequence,
        ballNumber,

```

```

        requirePerson(delivery.batter, 'batter'),
        requirePerson(delivery.non_striker, 'non-striker'),
        requirePerson(delivery.bowler, 'bowler'),
        delivery.runs.batter,
        delivery.runs.extras,
        delivery.runs.total,
        Boolean(delivery.runs.non_boundary),
        extras.wides ?? null,
        extras.noballs ?? null,
        extras.byes ?? null,
        extras.legbyes ?? null,
        extras.penalty ?? null,
        submissionId,
    ]
);

if (rows.length === 0) continue; // already present; nothing further to insert
const deliveryId = rows[0].delivery_id as number;
deliveryCount += 1;

for (const [wicketOrdinal, wicket] of (delivery.wickets ?? []).entries()) {
    const wicketId = await scalar<number>(
        `INSERT INTO delivery_wicket
            (delivery_id, ordinal, kind, source_kind, player_out_id)
            VALUES ($1,$2,$3,$4,$5) RETURNING wicket_id`,
        [
            deliveryId,
            wicketOrdinal,
            wicket.kind,
            wicket.kind,
            requirePerson(wicket.player_out, 'dismissed player'),
        ]
    );

    for (const [fielderOrdinal, fielder] of (wicket.fielders ?? []).entries()) {
        await client.query(
            `INSERT INTO delivery_wicket_fielder
                (wicket_id, ordinal, person_id, is_substitute)
                VALUES ($1,$2,$3,$4)`,
            [
                wicketId,
                fielderOrdinal,
                fielder.name ? (personId.get(fielder.name) ?? null) : null,
                Boolean(fielder.substitute),
            ]
        );
    }
}
}

```

```

    if (delivery.review) {
      const umpire = delivery.review.umpire
      ? await scalar<number | null>(
        'SELECT official_id FROM official WHERE display_name = $1',
        [delivery.review.umpire]
      ).catch(() => null)
      : null;
      await client.query(
        `INSERT INTO delivery_review
          (delivery_id, by_team_id, umpire_id, batter_id, decision, type)
        VALUES ($1,$2,$3,$4,$5,$6)`,
        [
          deliveryId,
          teamId.get(delivery.review.by),
          umpire,
          delivery.review.batter ? (personId.get(delivery.review.batter) ?? null)
          : delivery.review.decision,
          delivery.review.type ?? null,
        ]
      );
    }

    for (const kind of ['role', 'player'] as const) {
      for (const replacement of delivery.replacements?.[kind] ?? []) {
        await client.query(
          `INSERT INTO delivery_replacement
            (delivery_id, replacement_type, in_person_id, out_person_id, reason,
            VALUES ($1,$2,$3,$4,$5,$6)`,
          [
            deliveryId,
            kind,
            requirePerson(replacement.in, 'incoming replacement'),
            replacement.out ? (personId.get(replacement.out) ?? null) : null,
            replacement.reason ?? null,
            (replacement as { role?: string }).role ?? null,
          ]
        );
      }
    }
  }
}

await client.query('COMMIT');

console.log(`Ingested ${sourceRef}`);
console.log(` fixture_id      : ${fixtureId}`);
console.log(` people resolved : ${personId.size}`);
console.log(` deliveries added : ${deliveryCount}`);

```

```

    if (deliveryCount === 0) {
      console.log(' (already present: this run was a no-op, which is the intended behavior)');
    }
  }
}

main()
  .catch(async (error: unknown) => {
    await client.query('ROLLBACK').catch(() => undefined);
    console.error('Ingestion failed.');
```

console.error(error instanceof Error ? error.message : error);

```

    process.exitCode = 1;
  })
  .finally(() => {
    void client.end();
  });

```

I've also applied `requirePerson` to `wicket.player_out` and `replacement.in`, both of which are `NOT NULL` columns with the same silent-null problem. The nullable ones — fielders, review batter, replacement out — keep `personId.get`, since null is legitimate there.

Save it, then re-run the unregistered-name file:

```

powershell

npm run db:ingest
PS C:\Users\BenSwartz\Sport-Analytics-Tool> npm run db:ingest --
workspace=@sport-analytics/backend -- "C:\Users\BenSwartz\Sport-Analytics-
Tool\database\seeds\invalid\unregistered-name.json"
> @sport-analytics/backend@0.1.0 db:ingest
> tsx --env-file=.env scripts/ingest-match.ts C:\Users\BenSwartz\Sport-Analytics-
Tool\database\seeds\invalid\unregistered-name.json
Ingestion failed.
The batter "AN Unregistered" is absent from this match's registry. Every person
referenced by a delivery must appear in info.registry.people.
npm error Lifecycle script db:ingest failed with error:
npm error code 1
Show more
npm error path C:\Users\BenSwartz\Sport-Analytics-Tool\apps\backend
npm error workspace @sport-analytics/backend@0.1.0
npm error location C:\Users\BenSwartz\Sport-Analytics-Tool\apps\backend
npm error command failed
npm error command C:\windows\system32\cmd.exe /d /s /c tsx --env-file=.env
scripts/ingest-match.ts C:\Users\BenSwartz\Sport-Analytics-
Tool\database\seeds\invalid\unregistered-name.json
PS C:\Users\BenSwartz\Sport-Analytics-Tool>

```

That's the message a submitter should get — names the field, the value, and the rule.

Two checks before committing, because the change touched the valid path too.

Step 13 — confirm the seed still works and nothing was left behind by the failed runs:

```

powershell

```

```
npm run db:seed --workspace=@sport-analytics/backend
```

All four should report no-ops. If any reports deliveries added, a failed run left partial state and the rollback isn't doing its job — that would be a finding worth stopping on.

Then verify the delivery count is still exactly 955:

```
powershell
```

```
npm run db:sql --workspace=@sport-analytics/backend -- scripts/queries/state.sql
```

Paste both.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> npm run db:seed --
workspace=@sport-analytics/backend
> @sport-analytics/backend@0.1.0 db:seed
> tsx --env-file=.env scripts/seed.ts
Seeding 4 fixtures from C:\Users\BenSwartz\Sport-Analytics-
Tool\database\seeds\matches
--- 1399114.json ---
Ingested 1399114
  fixture_id    : 1
  people resolved : 28
  deliveries added : 0
  (already present: this run was a no-op, which is the intended behaviour)
--- 1462921.json ---
Ingested 1462921
  fixture_id    : 2
  people resolved : 26
  deliveries added : 0
  (already present: this run was a no-op, which is the intended behaviour)
--- 423788.json ---
Ingested 423788
  fixture_id    : 3
  people resolved : 27
  deliveries added : 0
  (already present: this run was a no-op, which is the intended behaviour)
--- 729307.json ---
Ingested 729307
  fixture_id    : 4
  people resolved : 27
  deliveries added : 0
  (already present: this run was a no-op, which is the intended behaviour)
Seed complete.
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
PS C:\Users\BenSwartz\Sport-Analytics-Tool> npm run db:sql --
workspace=@sport-analytics/backend -- scripts/queries/state.sql
> @sport-analytics/backend@0.1.0 db:sql
> tsx --env-file=.env scripts/sql.ts scripts/queries/state.sql
```

(index)	fixture_id	source_ref	live_deliveries
0	'1'	'1399114'	'241'
1	'2'	'1462921'	'216'
2	'3'	'423788'	'261'
3	'4'	'729307'	'237'

PS C:\Users\BenSwartz\Sport-Analytics-Tool>

Show more

Clean. Four fixtures, 955 deliveries, no partial rows from the three rejected submissions — the transaction rollback held under all three failure modes, which is worth more as evidence than the rejections themselves.

Step 14 — document the invalid examples. Create `database/seeds/invalid/README.md`:

powershell

@'

Invalid submission examples

Each file is match 729307 with exactly one deliberate defect introduced. They are not loaded by the seed. They exist to demonstrate that a defective submission is rejected with a message identifying the problem, and that a rejected submission leaves no partial data behind.

They are produced by `scripts/make\_invalid\_seeds.py` from the valid file under `../matches`, so the defect is the only difference.

File	Defect	Rejected by
`unregistered-name.json`	The batter on the first delivery is a name absent from `i	
`inconsistent-runs.json`	`runs.total` on the first delivery is 6 where batter plus	
`striker-equals-non-striker.json`	The non-striker on the first delivery is the sam	

Rejection happens at two layers: the application refuses input it cannot resolve, and the database refuses rows that violate its own rules. Neither layer is relied upon alone.

Verification

Each file was submitted to the ingestion script on 10 August 2026. Each was rejected with a message naming the defect, and the database was unchanged afterwards: four fixtures and 955 deliveries before and after.

AI Declaration

The preceding document was generated with the assistance of Claude-Web[Claude Opus 5].

@ | Set-Content -Path database\seeds\invalid\README.md -Encoding utf8

Then commit. You're on `feat/46-add-development-seed-data`:

powershell

git add database/seeds/invalid scripts/make_invalid_seeds.py apps/backend/scripts/inge

powershell

```
git commit -m "feat(db): add invalid submission examples and enforce registry resoluti
```

Producing them exposed a defect: the registry guard in ingest-match.ts was never calle

Verified: each example is rejected with a message identifying the defect, and the data

Refs #63

Assisted-by: Claude-Web[Claude Opus 5]"

```
powershell
```

```
git push
```

done

That leaves one criterion on #46: the ERD.

Step 15 — generate it from the live schema rather than drawing it, so it can't drift. Your docs already use Mermaid, so an `erDiagram` fits the existing tooling and renders on the published site without new dependencies.

Create `apps/backend/scripts/queries/erd.sql`:

```
sql

-- Foreign key relationships, for generating the ERD.
SELECT
    src.relname AS child_table,
    tgt.relname AS parent_table,
    con.conname AS constraint_name
FROM pg_constraint con
JOIN pg_class src ON src.oid = con.conrelid
JOIN pg_class tgt ON tgt.oid = con.confrelid
JOIN pg_namespace n ON n.oid = src.relnamespace
WHERE con.contype = 'f'
    AND n.nspname = 'public'
ORDER BY src.relname, tgt.relname;
```

Run it:

```
powershell
```

```
npm run db:sql --workspace=@sport-analytics/backend -- scripts/queries/erd.sql
```

Paste the output. From that I'll write the Mermaid diagram — the relationships come from the database rather than from my memory of the migration, which matters because a hand-written ERD that disagrees with the schema is worse than none.

One decision to make while that runs: a full ERD with every column would be enormous across 22 tables and unreadable on the docs site. I'd suggest relationships and keys only, with the column detail left to `schema.md`. Say if you'd rather have everything.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> npm
run db:sql --workspace=@
sport-analytics/backend -
scripts/queries/erd.sql >
@sport-analytics/backe...
```

PASTED

41 foreign keys, all accounted for. Relationships and keys only — the column detail belongs in `schema.md` and duplicating it would create two things that can disagree.

Step 16 — create `docs/database/erd.md`:

powershell

@'

Entity relationship diagram

The relationships below were read from the database with
'apps/backend/scripts/queries/erd.sql', which lists every foreign key in the
'public' schema. The diagram is therefore a description of the schema as
migrated, not a separate design document. Column detail is in
[the event model](schema.md).

Provenance and identity

```
```mermaid
erDiagram
 app_user ||--o{ submission : submits
 submission ||--o{ fixture : "first seen in"
 submission ||--o{ delivery : supplies
 person ||--o{ person_alias : "known as"
...

```

Every fixture and every delivery carries the submission it arrived in. A person is identified by their registry reference; the names they have appeared under are kept separately and are never a join key.

### ## Match structure

```
```mermaid
erDiagram
    competition ||--o{ fixture : contains
    venue ||--o{ fixture : hosts
    team ||--o{ fixture : "won, eliminated or won toss"
    fixture ||--o{ fixture_team : contests
    team ||--o{ fixture_team : "plays in"
    fixture ||--o{ fixture_squad : names
    person ||--o{ fixture_squad : "selected in"
    team ||--o{ fixture_squad : "selected for"
    fixture ||--o{ fixture_official : officiated
    official ||--o{ fixture_official : officiates
    fixture ||--o{ fixture_player_of_match : awards
    person ||--o{ fixture_player_of_match : "awarded to"
    fixture ||--o{ innings : "divided into"
    team ||--o{ innings : bats
    innings ||--o{ innings_powerplay : has
    innings ||--o{ innings_absent : records
    person ||--o{ innings_absent : absent
    innings ||--o{ innings_miscounted_over : records
...

```

A fixture holds any number of innings rather than two; a super over adds a third

and fourth. Officials are separate from persons because the source gives them no stable identifier.

Events

```
```mermaid
erDiagram
 innings ||--o{ delivery : contains
 delivery ||--o| delivery : supersedes
 person ||--o{ delivery : "faces, partners or bowls"
 delivery ||--o{ delivery_wicket : "may take"
 dismissal_kind ||--o{ delivery_wicket : classifies
 person ||--o{ delivery_wicket : "dismissed in"
 delivery_wicket ||--o{ delivery_wicket_fielder : credits
 person ||--o{ delivery_wicket_fielder : fields
 delivery ||--o| delivery_review : "may be reviewed"
 team ||--o{ delivery_review : reviews
 person ||--o{ delivery_review : "review concerns"
 official ||--o{ delivery_review : "decision reviewed"
 delivery ||--o{ delivery_replacement : records
 person ||--o{ delivery_replacement : "replaces or replaced"
```
```

The delivery is the event. A correction inserts a new row and marks the prior one superseded through `superseded\_by`, so the relationship from delivery to delivery is the correction history. All derivation reads `delivery\_current`, the view of rows not yet superseded.

AI Declaration

The preceding document was generated with the assistance of Claude-Web[Claude Opus 5].
'@ | Set-Content -Path docs\database\erd.md -Encoding utf8

Split into three diagrams because one graph of 22 tables and 41 edges is unreadable — but say if you'd rather have it whole.

Then add to `mkdocs.yml` nav, after the Event Model line:

```
yaml
- Entity Relationships: database/erd.md
```

Then verify it builds and the Mermaid parses:

```
powershell
python -m mkdocs build --strict
```

Paste the output.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> - Entity Relationships:
database/erd.md
At line:1 char:2
+ - Entity Relationships: database/erd.md
+ ~
Missing expression after unary operator '-'.
At line:1 char:3
+ - Entity Relationships: database/erd.md
+ ~~~~~
Unexpected token 'Entity' in expression or statement.
+ CategoryInfo          : ParserError: (:) [],
ParentContainsErrorRecordException
+ FullyQualifiedErrorId : MissingExpressionAfterOperator

PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

That YAML line goes into `mkdocs.yml` in your editor, not the terminal. My fault for not saying so.

Open `mkdocs.yml`, find:

```
yaml

- Event Model: database/schema.md
```

and add directly beneath it, at the same indentation:

```
yaml

- Entity Relationships: database/erd.md
```

Save, then run the build:

```
powershell

python -m mkdocs build --strict
```

Also confirm the `erd.md` file itself was created — that here-string contained nested triple-backtick fences, and PowerShell should handle them fine inside `@'...'@`, but worth checking:

```
powershell

Get-Content docs\database\erd.md -TotalCount 5
```

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> python -m mkdocs build --strict
| ⚠ Warning from the Material for MkDocs team
|
| MkDocs 2.0, the underlying framework of Material for MkDocs,
| will introduce backward-incompatible changes, including:
|
| × All plugins will stop working – the plugin system has been removed
| × All theme overrides will break – the theming system has been rewritten
| × No migration path exists – existing projects cannot be upgraded
| × Closed contribution model – community members can't report bugs
| × Currently unlicensed – unsuitable for production use
|
| Our full analysis:
|
| https://squidfunk.github.io/mkdocs-material/blog/2026/02/18/mkdocs-2.0/
INFO - Cleaning site directory
INFO - Building documentation to directory: C:\Users\BenSwartz\Sport-Analytics-Tool\site
INFO - The following pages exist in the docs directory, but are not included in the
"nav"
    configuration:
    - environment.md
    - adr\0003-azure-hosting.md
    - data\cricsheet.md
    - deployment\azure-backend.md
    - deployment\azure-fronted.md
    - deployment\cloudflare_pages.md
INFO - Documentation built in 4.61 seconds
PS C:\Users\BenSwartz\Sport-Analytics-Tool>

Show more
```

Strict build passes and `erd.md` isn't in the not-in-nav list, so the nav entry took.

The build can't tell you whether the Mermaid actually renders, though — MkDocs passes it through as a code fence and the browser draws it. A syntax error in an `erDiagram` shows as a blank space on the page, not a build failure. Worth checking before you commit:

```
powershell
start site\database\erd\index.html
```

Look for three diagrams. If any renders as raw text or an error box, tell me which and paste the section.

Meanwhile, `site/` is dirty again from the build — discard it before committing:

```
powershell
git checkout -- site/
```

Then:

powershell

```
git add docs/database/erd.md mkdocs.yml apps/backend/scripts/queries/erd.sql
git commit -m "docs(db): add an entity relationship diagram generated from the schema"
```

Refs #46

Assisted-by: Claude-Web[Claude Opus 5]"

```
git push
```

Hold the commit until you've confirmed the diagrams render.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> git add docs/database/erd.md
mkdocs.yml apps/backend/scripts/queries/erd.sql
>> git commit -m "docs(db): add an entity relationship diagram generated from the
schema" -m "The relationships are read from pg_constraint rather than drawn by
hand, so the diagram describes the schema as migrated. Split into provenance,
match structure and events, because one graph of 22 tables and 41 foreign keys is
unreadable.
>>
>> Refs #46
>> Assisted-by: Claude-Web[Claude Opus 5]"
>> git push
warning: in the working copy of 'apps/backend/scripts/queries/erd.sql', CRLF will be
replaced by LF the next time Git touches it
warning: in the working copy of 'docs/database/erd.md', CRLF will be replaced by LF
the next time Git touches it
[feat/46-add-development-seed-data 363832f] docs(db): add an entity relationship
diagram generated from the schema
3 files changed, 93 insertions(+)
create mode 100644 apps/backend/scripts/queries/erd.sql
create mode 100644 docs/database/erd.md
Enumerating objects: 19, done.
Counting objects: 100% (19/19), done.
Delta compression using up to 12 threads
Compressing objects: 100% (11/11), done.
Writing objects: 100% (11/11), 2.40 KiB | 818.00 KiB/s, done.
Total 11 (delta 6), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a new pull request for 'feat/46-add-development-seed-data':
remote: https://sdp.ms.wits.ac.za/git-push-pray/Sport-Analytics-Tool/pulls/new/feat/46-add-development-seed-data
remote:
remote: . Processing 1 references
remote: Processed 1 references in total
To https://sdp.ms.wits.ac.za/git-push-pray/Sport-Analytics-Tool.git
3944ed9..363832f feat/46-add-development-seed-data -> feat/46-add-
development-seed-data
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

Committed and pushed. You skipped the render check, though — do it now, because a broken `erDiagram` shows as blank space on the published site and the build won't catch it:

```
powershell
```

```
start site\database\erd\index.html
```

Three diagrams should draw. If any is blank or shows an error box, say which and I'll fix the syntax before the PR goes up.

Assuming they render, open the PR:

Title: `#46 Add development seed data and entity relationship diagram`

```
markdown
```

Purpose

Completes the outstanding parts of #46 and #63 following the schema work in #27: repeatable development seed data, an entity relationship diagram, and invalid submission examples.

Changes

- Four Cricsheet match files under `database/seeds/matches`, loaded by the ingestion script rather than by SQL inserts, so the seed exercises the same validation path as a real submission and cannot drift from the schema. The fixtures cover the reference scorecard, a super over, and both source representations of a miscounted over ball count.
- `db:seed`, which runs the ingester over those files.
- Three invalid submission examples under `database/seeds/invalid`, each match 729307 with one deliberate defect, demonstrating rejection at both the application and database layers.
- A fix to `ingest-match.ts`: the registry guard was never called, so a name absent from `info.registry.people` was inserted as null and rejected by a not-null constraint with a message that said nothing about the real problem. Names now resolve through a check that names the field, the value and the rule.
- `docs/database/erd.md`, with the relationships read from `pg\_constraint` rather than drawn by hand.
- The Cricsheet licence and required attribution, recorded in `database/seeds/README.md`.

Testing

First seed run loads 955 deliveries across four fixtures; second run adds zero. Each invalid example is rejected with a message identifying the defect, and the database is unchanged afterwards. `mkdocs build --strict` passes.

Known Limitations

The seed depends on the four match files being committed, which adds 346 kB to the repository. The alternative was committing generated SQL, which would go stale against the schema.

The ERD shows relationships and keys only. Column detail remains in `docs/database/schema.md`.

Related Issues

Refs #46

Refs #63

Refs rather than **Closes** on both — #63 still wants its expected figures packaged as a test fixture rather than a markdown file, and #46's remaining criteria include the PR review itself.

the render worked, now what?

Open the PR with the title and description from my last message, then request a review — Shayna or Dean, whoever's around.

While that's waiting, the four #27 follow-ups are still unrecorded, and one of them is the super-over exclusion. That's the two-minute job I flagged earlier and it's now more urgent, not less: your seed has a super over in it, so whoever writes derivation will be computing over data that includes those four innings from the start.

Want me to draft that one issue? The other three can wait.

This pull request has changes conflicting with the target branch.

- apps/backend/scripts/ingest-match.ts

Someone changed `ingest-match.ts` in `main` while you were working. Given #27 merged your version, this is a later edit from another member — worth knowing what they did before resolving.

```
powershell  
  
git fetch origin  
git log --oneline origin/main -5 -- apps/backend/scripts/ingest-match.ts
```

Paste that. Then start the merge:

```
powershell  
  
git merge origin/main
```

It will stop with the conflict. Don't resolve blind — paste me the conflicted region:

```
powershell  
  
Select-String -Path apps\backend\scripts\ingest-match.ts -Pattern "<<<<<<|=====|>>>>>>"
```

Both outputs, then I'll tell you what to keep. Your changes are the `ballNumber` derivation and `requirePerson`; whatever theirs are, they'll need to survive too rather than being overwritten.

```

PS C:\Users\BenSwartz\Sport-Analytics-Tool> git fetch origin
>> git log --oneline origin/main -5 -- apps/backend/scripts/ingest-match.ts
remote: Enumerating objects: 523, done.
remote: Counting objects: 100% (454/454), done.
remote: Compressing objects: 100% (369/369), done.
Receiving objects: 63% (238/377), 2.89 MiB | 363.00 KiB/sremote: Total 377 (delta
203), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (377/377), 3.10 MiB | 363.00 KiB/s, done.
Resolving deltas: 100% (203/203), completed with 34 local objects.
From https://sdp.ms.wits.ac.za/git-push-pray/Sport-Analytics-Tool
* [new branch] chore/98-format-issue-markdown -> origin/chore/98-format-
Show more
issue-markdown
* [new branch] docs/47-public-read-openapi-sync -> origin/docs/47-public-
read-openapi-sync
* [new branch] docs/59-openapi-baseline -> origin/docs/59-openapi-baseline
* [new branch] feat/47-public-read-api -> origin/feat/47-public-read-api
* [new branch] feat/62-database-repositories -> origin/feat/62-database-
repositories
* [new branch] feat/64-frontend-auth-state -> origin/feat/64-frontend-auth-
state
* [new branch] feat/89-public-landing-shell -> origin/feat/89-public-landing-
shell
bca3103..489e885 main -> origin/main
db4002c (origin/chore/98-format-issue-markdown) chore(format): establish clean
Prettier baseline
8e35222 fix(ingest): populate ball number from legal ball count
000f729 feat(db): validate the event schema against a published scorecard
PS C:\Users\BenSwartz\Sport-Analytics-Tool>

```

It's a Prettier reformat — `chore(format): establish clean Prettier baseline` touched the whole file. So the conflict is formatting versus your logic changes, not competing logic.

Also worth noting: `feat/62-database-repositories` and `feat/47-public-read-api` now exist, so the repository layer and public read API are being worked on. That's the thing Dean was asking about this morning.

Merge and look at the damage:

powershell

```
git merge origin/main
```

powershell

```
Select-String -Path apps\backend\scripts\ingest-match.ts -Pattern "<<<<<<<" | Measure-
```

Paste both. If Prettier rewrote the whole file, you may have conflicts throughout rather than in one place — in which case the cleaner resolution is to take their formatted version wholesale and reapply your two changes on top, rather than picking through dozens of hunks. Tell me the count and I'll say which way to go.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> git merge origin/main
Auto-merging apps/backend/package.json
Auto-merging apps/backend/scripts/ingest-match.ts
CONFLICT (content): Merge conflict in apps/backend/scripts/ingest-match.ts
Auto-merging mkdocs.yml
Automatic merge failed; fix conflicts and then commit the result.
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

Only `ingest-match.ts` conflicted; `package.json` and `mkdocs.yml` auto-merged. Count the conflict regions:

```
powershell
Select-String -Path apps\backend\scripts\ingest-match.ts -Pattern "<<<<<<" | Measure-
```

Paste that. If it's one or two, we resolve in place. If it's a dozen, take their formatted file and reapply your changes:

```
powershell
git checkout --theirs apps/backend/scripts/ingest-match.ts
```

then re-add `requirePerson` and the `ballNumber` derivation on top. Tell me the count first.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> Select-String -Path
apps\backend\scripts\ingest-match.ts -Pattern "<<<<<<" | Measure-Object |
Select-Object Count
Count
-----
2
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

Two regions — manageable in place. Show me both:

```
powershell
Select-String -Path apps\backend\scripts\ingest-match.ts -Pattern "<<<<<<|=====|>>>>>>"
```

Paste the output and I'll tell you exactly what the resolved text should be.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> Select-String -Path apps\backend\scripts\ingest-match.ts -Pattern "<<<<<<|===|>>>>>>" -Con...
```

PASTED

Both conflicts are yours-versus-theirs on the same lines, and yours is the correct side — theirs is just the pre-fix code that Prettier happened to reformat. Take yours in both, keeping Prettier's trailing-comma style.

Conflict 1 — replace lines 342 to 350 (the whole <<<<<< through >>>>>> block) with:

```
requirePerson(delivery.batter, 'batter'),
requirePerson(delivery.non_striker, 'non-striker'),
requirePerson(delivery.bowler, 'bowler'),
```

Conflict 2 — replace lines 373 to 383 with:

```
[
  deliveryId,
  wicketOrdinal,
  wicket.kind,
  wicket.kind,
  requirePerson(wicket.player_out, 'dismissed player'),
],
```

Note the trailing comma after the closing bracket — theirs has it, yours didn't, and Prettier will want it.

Then check nothing else of yours was lost in the auto-merge. Prettier reformatted the whole file, so `requirePerson`'s definition and the `ballNumber` derivation should have survived, but verify:

powershell

```
Select-String -Path apps\backend\scripts\ingest-match.ts -Pattern "function requirePer
```

Paste that after resolving.

can you do it from the terminal?

Sed-style edits on conflict markers from PowerShell are fiddly and easy to get subtly wrong, and this file is your merged migration work — I'd rather not risk a silent mangle. But there's a clean terminal route that doesn't involve hand-editing:

```
powershell  
git checkout --ours apps/backend/scripts/ingest-match.ts
```

That takes your entire version of the file, discarding theirs. Since the only difference in theirs is Prettier formatting — no logic changes — you lose the formatting and keep all your logic. Then let Prettier put the formatting back:

```
powershell  
npx prettier --write apps/backend/scripts/ingest-match.ts
```

That should land you on exactly the same result as resolving by hand: your logic, their formatting.

Then verify:

```
powershell  
Select-String -Path apps\backend\scripts\ingest-match.ts -Pattern "<<<<<<|function re
```

Expecting no conflict markers, one `requirePerson` definition, one `ballNumber`.

Paste that, then:

```
powershell  
git add apps/backend/scripts/ingest-match.ts  
git commit
```

`:wq` on the default merge message, then `git push`.

done

"Done" without the verification output — I'd rather confirm than assume, since this file is the one carrying your ingestion logic:

```
powershell  
Select-String -Path apps\backend\scripts\ingest-match.ts -Pattern "<<<<<<|function re  
git status -sb
```

And the strongest check is behavioural rather than textual — run the seed and one invalid example:

powershell

```
npm run db:seed --workspace=@sport-analytics/backend
npm run db:ingest --workspace=@sport-analytics/backend -- "C:\Users\BenSwartz\Sport-An
```

Four no-ops and the registry error message means the merge preserved everything. Paste all of it.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> Select-String -Path
apps\backend\scripts\ingest-match.ts -Pattern "<<<<<<|function
requirePerson|const ballNumber"
>> git status -sb
apps\backend\scripts\ingest-match.ts:107: function requirePerson(name: string,
role: string):
number {
apps\backend\scripts\ingest-match.ts:322:    const ballNumber =
    ${over.over}.${Math.max(legalBalls, 1)};
## feat/46-add-development-seed-data...origin/feat/46-add-development-seed-
data
Show more
?? site/adr/
?? site/ai_policy.pdf
?? site/architecture/system-architecture/
?? site/data/
?? site/database/erd/
?? site/deployment/azure-backend/
?? site/deployment/azure-fronted/
?? site/deployment/cloudflare_pages/
?? site/design/
?? site/environment/
?? site/planning/
?? site/security/auth-provider-comparison/
?? site/security/authentication/
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

No conflict markers, both changes present, working tree clean apart from **site/** build noise. The branch shows level with origin, so the merge commit is pushed.

Now the behavioural check — that's the one that actually proves the merge didn't break anything:

powershell

```
npm run db:seed --workspace=@sport-analytics/backend
```

powershell

```
npm run db:ingest --workspace=@sport-analytics/backend -- "C:\Users\BenSwartz\Sport-An
```

Paste both. Four no-ops, then the registry error naming the batter.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> npm run db:seed --
workspace=@sport-analytics/backend
> @sport-analytics/backend@0.1.0 db:seed
> tsx --env-file=.env scripts/seed.ts
Seeding 4 fixtures from C:\Users\BenSwartz\Sport-Analytics-
Tool\database\seeds\matches
--- 1399114.json ---
Ingested 1399114
  fixture_id    : 1
  people resolved : 28
  deliveries added : 0
  (already present: this run was a no-op, which is the intended behaviour)
--- 1462921.json ---
Ingested 1462921
  fixture_id    : 2
  people resolved : 26
  deliveries added : 0
  (already present: this run was a no-op, which is the intended behaviour)
--- 423788.json ---
Ingested 423788
  fixture_id    : 3
  people resolved : 27
  deliveries added : 0
  (already present: this run was a no-op, which is the intended behaviour)
--- 729307.json ---
Ingested 729307
  fixture_id    : 4
  people resolved : 27
  deliveries added : 0
  (already present: this run was a no-op, which is the intended behaviour)
Seed complete.
PS C:\Users\BenSwartz\Sport-Analytics-Tool> npm run db:ingest --
workspace=@sport-analytics/backend -- "C:\Users\BenSwartz\Sport-Analytics-
Tool\database\seeds\invalid\unregistered-name.json"
> @sport-analytics/backend@0.1.0 db:ingest
> tsx --env-file=.env scripts/ingest-match.ts C:\Users\BenSwartz\Sport-Analytics-
Tool\database\seeds\invalid\unregistered-name.json
Ingestion failed.
The batter "AN Unregistered" is absent from this match's registry. Every person
referenced by a delivery must appear in info.registry.people.
npm error Lifecycle script db:ingest failed with error:
npm error code 1
npm error path C:\Users\BenSwartz\Sport-Analytics-Tool\apps\backend
npm error workspace @sport-analytics/backend@0.1.0
npm error location C:\Users\BenSwartz\Sport-Analytics-Tool\apps\backend
npm error command failed
npm error command C:\windows\system32\cmd.exe /d /s /c tsx --env-file=.env
```

```
scripts/ingest-match.ts C:\Users\BenSwartz\Sport-Analytics-Tool\database\seeds\invalid\unregistered-name.json
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

Show more

Merge is sound — seed idempotent, registry check intact.

The PR should be mergeable now. Two things while it waits for review:

Run `npm run check` before asking anyone to look. The Prettier baseline just landed in `main` and your new files — `seed.ts`, `sql.ts`, `erd.sql`, the queries — were written before it. If they fail the format check, better you find it than a reviewer:

```
powershell
```

```
npm run check
```

Then the super-over issue. Still the one piece of knowledge that exists only here, and now there's a super over sitting in the seed that everyone will develop against.

Paste the check output and I'll draft the issue while you request the review.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> npm run check
> sport-analytics-tool@0.1.0 check
> npm run structure:check && npm run format:check && npm run lint && npm run
typecheck && npm run test && npm run openapi:lint && npm run build
> sport-analytics-tool@0.1.0 structure:check
> node scripts/check-required-files.mjs
Repository structure check passed (20 required files).
> sport-analytics-tool@0.1.0 format:check
> prettier --check .
Checking formatting...
[warn] apps/backend/scripts/seed.ts
[warn] database/seeds/invalid/inconsistent-runs.json
[warn] database/seeds/invalid/README.md
[warn] database/seeds/invalid/striker-equals-non-striker.json
[warn] database/seeds/invalid/unregistered-name.json
[warn] database/seeds/matches/1399114.json
[warn] database/seeds/matches/1462921.json
[warn] database/seeds/matches/423788.json
[warn] database/seeds/matches/729307.json
[warn] database/seeds/README.md
[warn] docs/database/erd.md
[warn] Code style issues found in 11 files. Run Prettier with --write to fix.
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

Good catch — and one of those you must **not** fix.

Prettier wants to reformat the eight seed match JSON files. Do not let it. Those are third-party source data, committed so the seed loads real submissions; reformatting them means the

bytes no longer match what Cricsheet published, and `ingest-match.ts` hashes the file into `submission.source_sha256`. Reformatting silently changes the provenance hash of every seeded submission.

So: exclude the data from Prettier, format the rest.

Step 1 — add to `.prettierignore` (create it if absent) in the repository root:

```
# Third-party source data. These files are committed verbatim so that the
# submission hash matches what the source published. They must not be reformatted.
database/seeds/matches/
database/seeds/invalid/*.json
```

Step 2 — format the files that are genuinely ours:

```
powershell
npx prettier --write apps/backend/scripts/seed.ts database/seeds/README.md database/se
```

Step 3 — re-run:

```
powershell
npm run check
```

Paste the result. Note it'll now run past format into lint, typecheck, test, openapi:lint and build — so it may surface something else, and it'll take a while.

Worth flagging for the PR description too: this is a genuine constraint the repository didn't previously have, and the `.prettierignore` reason should be visible to reviewers rather than looking like someone dodging the formatter.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> npm run check
> sport-analytics-tool@0.1.0 check
> npm run structure:check && npm run format:check && npm run lint && npm run
typecheck && npm run test && npm run openapi:lint && npm run build
> sport-analytics-tool@0.1.0 structure:check
> node scripts/check-required-files.mjs
Repository structure check passed (20 required files).
> sport-analytics-tool@0.1.0 format:check
> prettier --check .
Checking formatting...
All matched files use Prettier code style!
> sport-analytics-tool@0.1.0 lint
> npm run lint --workspaces --if-present
> @sport-analytics/backend@0.1.0 lint
> eslint src tests --ext .ts
> @sport-analytics/frontend@0.1.0 lint
> eslint src --ext .ts,tsx
```

```
> @sport-analytics/contracts@0.1.0 lint
> eslint src --ext .ts
> sport-analytics-tool@0.1.0 typecheck
> npm run typecheck --workspaces --if-present
> @sport-analytics/backend@0.1.0 typecheck
> tsc --noEmit
src/auth/supabase-auth.ts:1:30 - error TS2307: Cannot find module
'@supabase/supabase-js' or its corresponding type declarations.
1 import { createClient } from '@supabase/supabase-js';
    ~~~~~

src/modules/public-read/public-read.controller.ts:7:8 - error TS2307: Cannot find
module '@sport-analytics/contracts' or its corresponding type declarations.
7 } from '@sport-analytics/contracts';
    ~~~~~

src/modules/public-read/public-read.service.ts:12:8 - error TS2307: Cannot find
module '@sport-analytics/contracts' or its corresponding type declarations.
12 } from '@sport-analytics/contracts';
    ~~~~~

Found 3 errors in 3 files.
Errors Files
  1 src/auth/supabase-auth.ts:1
  1 src/modules/public-read/public-read.controller.ts:7
  1 src/modules/public-read/public-read.service.ts:12
npm error Lifecycle script `typecheck` failed with error:
npm error code 2
npm error path C:\Users\BenSwartz\Sport-Analytics-Tool\apps\backend
npm error workspace @sport-analytics/backend@0.1.0
npm error location C:\Users\BenSwartz\Sport-Analytics-Tool\apps\backend
npm error command failed
npm error command C:\windows\system32\cmd.exe /d /s /c tsc --noEmit
> @sport-analytics/frontend@0.1.0 typecheck
> tsc --noEmit
> @sport-analytics/contracts@0.1.0 typecheck
> tsc --noEmit
PS C:\Users\BenSwartz\Sport-Analytics-Tool>

Show more
```

Format and lint pass. The three typecheck errors are not yours — they're in `src/auth/` and `src/modules/public-read/`, files that arrived in `main` from the auth and public-read API work. Almost certainly a missing `npm install` on your side after those branches added dependencies:

```
powershell

npm install
```

Then:

```
powershell

npm run check
```

If the errors persist after install, they're real and `main` is broken — `@sport-analytics/contracts` failing to resolve from the backend would mean the workspace link or the contracts build is broken, which would affect everyone. Worth knowing which, so paste the result either way.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> npm run check
> sport-analytics-tool@0.1.0 check
> npm run structure:check && npm run format:check && npm run lint && npm run
typecheck && npm run test && npm run openapi:lint && npm run build
> sport-analytics-tool@0.1.0 structure:check
> node scripts/check-required-files.mjs
Repository structure check passed (20 required files).
> sport-analytics-tool@0.1.0 format:check
> prettier --check .
Checking formatting...
All matched files use Prettier code style!
Show more
> sport-analytics-tool@0.1.0 lint
> npm run lint --workspaces --if-present
> @sport-analytics/backend@0.1.0 lint
> eslint src tests --ext .ts
> @sport-analytics/frontend@0.1.0 lint
> eslint src --ext .ts,.tsx
> @sport-analytics/contracts@0.1.0 lint
> eslint src --ext .ts
> sport-analytics-tool@0.1.0 typecheck
> npm run typecheck --workspaces --if-present
> @sport-analytics/backend@0.1.0 typecheck
> tsc --noEmit
src/modules/public-read/public-read.controller.ts:7:8 - error TS2307: Cannot find
module '@sport-analytics/contracts' or its corresponding type declarations.
7 } from '@sport-analytics/contracts';
~~~~~
src/modules/public-read/public-read.service.ts:12:8 - error TS2307: Cannot find
module '@sport-analytics/contracts' or its corresponding type declarations.
12 } from '@sport-analytics/contracts';
~~~~~
Found 2 errors in 2 files.
Errors Files
   1 src/modules/public-read/public-read.controller.ts:7
   1 src/modules/public-read/public-read.service.ts:12
npm error Lifecycle script `typecheck` failed with error:
npm error code 2
npm error path C:\Users\BenSwartz\Sport-Analytics-Tool\apps\backend
npm error workspace @sport-analytics/backend@0.1.0
npm error location C:\Users\BenSwartz\Sport-Analytics-Tool\apps\backend
npm error command failed
npm error command C:\windows\system32\cmd.exe /d /s /c tsc --noEmit
> @sport-analytics/frontend@0.1.0 typecheck
> tsc --noEmit
> @sport-analytics/contracts@0.1.0 typecheck
> tsc --noEmit
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

`npm install` fixed the Supabase one. The remaining two are different: `@sport-analytics/contracts` is a workspace package, so it resolves through the link — but the backend imports its *types*, and those only exist once the package has been built.

powershell

```
npm run build --workspace=@sport-analytics/contracts
npm run typecheck --workspace=@sport-analytics/backend
```

If that clears it, the finding is that `npm run check` typechecks the backend before building contracts, so a fresh clone fails on a step order rather than a real defect. Worth reporting on the public-read issue — CI would hit it too, if there were runners.

If it doesn't clear it, `main` genuinely doesn't typecheck and that's a bigger conversation.

Paste the result.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> npm run build --workspace=@sport-
analytics/contracts
>> npm run typecheck --workspace=@sport-analytics/backend
> @sport-analytics/contracts@0.1.0 build
> tsc -p tsconfig.json
> @sport-analytics/backend@0.1.0 typecheck
> tsc --noEmit
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

Confirmed: `main` typechecks fine, but only after contracts is built. `npm run check` runs typecheck before build, so on a fresh clone it fails — which is exactly what happened to you, and would happen to any teammate or CI runner.

That's a real finding and not yours to fix. Report it on whichever issue owns the public-read work, or raise it separately.

Finish your own check first:

powershell

```
npm run check
```

It should now get past typecheck into test, openapi:lint and build. Paste the result — if anything else fails I want to see whether it's yours before you request review.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> npm run check > sport-analytics-tool@0.1.0 check > npm run structure:check && npm run format:check ...
```

PASTED

Not yours. Every one of the 17 failures is the same line in `beforeEach` — `window.localStorage.clear is not a function` — in frontend test files from the auth and public-read work. Your changes touch nothing in `apps/frontend`.

The cause is a test-environment setup problem: something is stubbing `window.localStorage` with an object that lacks `clear`, or the jsdom environment isn't providing it. Either way it's in the frontend test setup, and it means `main` currently fails `npm run check` — which is the gate your merge requirements name.

Two things follow:

Your PR is fine to review. Say plainly in the description that `npm run check` fails on `main` for reasons unrelated to this branch, and name them: the contracts build ordering, and the frontend localStorage failures. A reviewer running `check` will hit both and needs to know they're pre-existing.

Tell the team now. `main` failing `check` is worse than any single issue on the board — it means nobody can use the gate to judge their own work, and with no CI runners there's nothing else catching it. Whoever owns the frontend auth tests should know today.

Want me to draft that message?

not now, nadav closed the PR, what next?

"Closed" and "merged" are different in Gitea, and it matters — a closed-unmerged PR means your work isn't in `main`.

Check which happened:

```
powershell  
  
git fetch origin  
git log --oneline origin/main -3
```

If your seed commits are in that list, it was merged. If not, it was closed without merging and you need to ask Nadav why — that's three commits of work sitting on a branch.

Paste the output.

i meant merged

Then finish the workflow — your methodology's post-merge steps:

```
powershell  
  
git switch main  
git pull origin main  
git branch -d feat/46-add-development-seed-data
```

In Gitea: delete the remote branch, and update the board — but **don't move #46 or #63 to Done**. Neither is complete. #46 still wants its acceptance criteria ticked and reviewed as a whole; #63 still wants the 729307 expected figures packaged as a test fixture rather than a markdown file. Both stay in progress with the delivered criteria ticked.

Then, in priority order:

The super-over issue. Two minutes, still unwritten, and now there's a super over in the seed that everyone develops against.

Tell the team `main` fails `npm run check`. Seventeen frontend test failures plus the contracts build ordering. With no CI runners, that gate is the only thing standing between the team and a broken `main`, and right now it's red for everyone.

The second is more urgent than it sounds — people will start ignoring `check` failures if they assume they're pre-existing, and then a real one slips through.

Which do you want first?

just tell me what to do step by step

Step 1 — clean up locally:

```
powershell  
  
git switch main  
git pull origin main  
git branch -d feat/46-add-development-seed-data
```

Paste the output.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> git switch main  
>> git pull origin main
```

```
>> git branch -d feat/46-add-development-seed-data
error: Your local changes to the following files would be overwritten by checkout:
    .prettierignore
    apps/backend/scripts/seed.ts
    database/seeds/README.md
    database/seeds/invalid/README.md
    docs/database/erd.md
Please commit your changes or stash them before you switch branches.
Aborting
remote: Failed to authenticate user
fatal: Authentication failed for 'https://sdp.ms.wits.ac.za/git-push-pray/Sport-
Analytics-Tool.git/'
error: cannot delete branch 'feat/46-add-development-seed-data' used by
worktree at 'C:/Users/BenSwartz/Sport-Analytics-Tool'
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

Show more

Uncommitted changes on your branch — the `.prettierignore` and the Prettier reformatting from the `npm run check` fixes. Those were never committed, so they weren't in the merged PR.

That means **the merged PR still fails** `format:check` on those files.

See what's actually different:

```
powershell

git status -sb
git diff --stat
```

Paste both. Then we commit them to a small follow-up branch rather than losing them — `.prettierignore` in particular matters, because without it the next person to run `prettier --write` will reformat the Cricsheet match files and silently change their submission hashes.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> git status -sb
>> git diff --stat
## feat/46-add-development-seed-data...origin/feat/46-add-development-seed-
data
M .prettierignore
M apps/backend/scripts/seed.ts
M database/seeds/README.md
M database/seeds/invalid/README.md
M docs/database/erd.md
?? site/adr/
?? site/ai_policy.pdf
?? site/architecture/system-architecture/
?? site/data/
?? site/database/erd/
?? site/deployment/azure-backend/
```

```
?? site/deployment/azure-fronted/
?? site/deployment/cloudflare_pages/
?? site/design/
?? site/environment/
?? site/planning/
?? site/security/auth-provider-comparison/
?? site/security/authentication/
warning: in the working copy of 'apps/backend/scripts/seed.ts', CRLF will be
replaced by LF the next time Git touches it
warning: in the working copy of 'database/seeds/README.md', CRLF will be
replaced by LF the next time Git touches it
warning: in the working copy of 'database/seeds/invalid/README.md', CRLF will
be replaced by LF the next time Git touches it
warning: in the working copy of 'docs/database/erd.md', CRLF will be replaced by
LF the next time Git touches it
.prettierrignore      | 4 ++++
apps/backend/scripts/seed.ts | 4 +--
database/seeds/README.md   | 15 ++++++-----
database/seeds/invalid/README.md | 13 ++++++-----
docs/database/erd.md      | 3 ++-
5 files changed, 23 insertions(+), 16 deletions(-)
PS C:\Users\BenSwartz\Sport-Analytics-Tool>

Show more
```

Small and clearly the formatting fixes plus `.prettierrignore`. Note `.prettierrignore` shows as modified, not new — it already existed and you appended to it.

Step 1 — commit them on the branch that's already merged, then open a small follow-up PR. Simplest, since the branch still exists and the changes belong with that work:

```
powershell
```

```
git add .prettierrignore apps/backend/scripts/seed.ts database/seeds/README.md database
```

```
powershell
```

```
git commit -m "chore(format): apply Prettier and exclude source match data" -m "The se
```

```
The Cricsheet match files are excluded from Prettier rather than reformatted. They are
```

```
Refs #46
```

```
Assisted-by: Claude-Web[Claude Opus 5]"
```

```
powershell
```

```
git push
```

Paste the output. Gitea may reopen the merged PR or need a new one — tell me which and I'll give you the description.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> git push
Enumerating objects: 29, done.
Counting objects: 100% (29/29), done.
Delta compression using up to 12 threads
Compressing objects: 100% (15/15), done.
Writing objects: 100% (15/15), 1.79 KiB | 262.00 KiB/s, done.
Total 15 (delta 11), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a new pull request for 'feat/46-add-development-seed-data':
remote: https://sdp.ms.wits.ac.za/git-push-pray/Sport-Analytics-Tool/pulls/new/feat/46-add-development-seed-data
Show more
remote:
remote: . Processing 1 references
remote: Processed 1 references in total
To https://sdp.ms.wits.ac.za/git-push-pray/Sport-Analytics-Tool.git
    8f55774..9b96bac feat/46-add-development-seed-data -> feat/46-add-development-seed-data
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

Pushed. Gitea is offering a new PR, so the merged one didn't reopen.

Step 2 — open the PR at that URL.

Title: #46 Apply Prettier to seed files and exclude source match data

markdown

Purpose

The files added in the previous pull request were written before the Prettier baseline landed in `main` and do not pass `format:check`. This applies Prettier to them and excludes the source match data from the formatter.

Changes

- Prettier applied to `seed.ts`, both seed READMEs and `erd.md`.
- `.prettierignore` extended to exclude `database/seeds/matches/` and the invalid example JSON files.

Testing

`npm run structure:check`, `format:check` and `lint` pass.

Known Limitations

The Cricsheet match files are deliberately excluded from Prettier rather than reformatted. They are committed verbatim so that the SHA-256 recorded in `submission.source\_sha256` matches the bytes the source published; reformatting them would silently change the provenance hash of every seeded submission.

`npm run check` does not currently pass on `main`, for reasons unrelated to this branch: the backend typecheck fails unless `@sport-analytics/contracts` is built first, and seventeen frontend tests fail in `beforeEach` with `window.localStorage.clear` is not a function`.

Related Issue

Refs #46

That last paragraph is the important one. It puts the broken `main` in front of a reviewer without you having to raise it separately — and it protects you, since anyone running `check` against your branch will hit both failures.

Request a review, then Step 3 is the super-over issue.

done

Step 3 — the super-over issue. Create it in Gitea, type **Task**, and paste this:

markdown

Description

Exclude super-over innings from batting and bowling aggregates.

The schema records super overs correctly: `innings.is\_super\_over` is set, and a fixture with a super over holds four innings rather than two. Nothing yet acts on that flag. Any aggregate written over `delivery\_current` without filtering it will include super-over deliveries.

Motivation

Super-over runs, wickets and balls do not count towards batting or bowling records in cricket. A career strike rate computed over unfiltered deliveries is wrong for every player who has batted in one.

This is difficult to catch later. Published scorecards exclude super-over figures too, so an aggregate that wrongly includes them will not obviously disagree with a scorecard unless the reference fixture happens to contain a super over. The error is silent.

99 matches in the corpus carry a super over, covering 204 innings. One of them, 423788, is in the development seed, so every developer is working against data that contains the case.

Acceptance Criteria

- [] Every batting and bowling aggregate excludes innings where `is\_super\_over` is true
- [] The exclusion is applied in one place rather than repeated per statistic
- [] A test computes an aggregate over a fixture containing a super over and compares it against the published scorecard figure
- [] The rule is documented in the derivation documentation, with the reason
- [] Any aggregate that legitimately includes super-over data, if one is identified, is named explicitly rather than filtered by default

Dependencies

Blocks the statistic derivation work. Should be resolved before aggregates are written rather than retrofitted.

Evidence

Test output against a super-over fixture and the published scorecard it is compared to.

AI Declaration: The preceding issue was generated with the assistance of Claude-Web[Cl

Labels: `type: feature` or `area: data`, `priority: high`. Milestone: Sprint 2 — it's derivation work, not Sprint 1. Add it to the project board in `Backlog`.

Don't self-assign. It belongs to whoever takes derivation, and the point is that they see it before they start.

Tell me when it's up and I'll give you Step 4.

what's the titlke?

Exclude super-over innings from batting and bowling aggregates

I can't assign to sprint 2 yet, hasnt been created

Leave the milestone empty then, and put it in `Backlog` on the board. Your Definition of Ready requires a milestone before it can move to `Ready`, so it's correctly parked until Sprint 2 exists.

Worth mentioning at Thursday's standup that only Sprint 1 has a milestone — Shayna's backlog document maps work to Sprints 2 and 3, but without the Gitea milestones there's nowhere to put it, and issues will accumulate unassigned to any sprint.

Tell me when the issue is up.

done

Step 4 — tell the team `main` fails `npm run check`.

Two independent failures, neither yours, both affecting everyone:

- Backend typecheck fails unless `@sport-analytics/contracts` is built first. `npm run check` runs typecheck before build, so a fresh clone fails on step order.
- Seventeen frontend tests fail in `beforeEach` with `window.localStorage.clear is not a function`, in `App.test.tsx` and `PublicBrowsePages.test.tsx`.

This matters more than it looks. With no CI runners, `npm run check` is the only gate — your own methodology calls it the compensating control. If it's red for everyone, people stop reading it, and then a real failure goes through unnoticed.

Want it as a WhatsApp message to the group, or as issues in Gitea? I'd do both: message now so nobody wastes an hour debugging it themselves, issues so it's tracked. The frontend one is Dean's or Gabriel's territory; the contracts ordering belongs with whoever owns the public-read work.

give me whatsapp

A With workaround B Shorter

Heads up — npm run check is failing on main at the moment, two separate things, neither of them anyone's branch:

1. Backend typecheck fails with "Cannot find module '@sport-analytics/contracts'" unless you build contracts first. check runs typecheck before build so a fresh clone always fails here. Workaround for now: npm run build --workspace=@sport-analytics/contracts

2. 17 frontend tests fail in beforeEach with "window.localStorage.clear is not a function" — App.test.tsx and PublicBrowsePages.test.tsx. Something in the test setup is stubbing localStorage without a clear method.

Flagging because we've got no CI runners, so check is the only gate we have. If it's red by default we'll all start ignoring it and something real will slip through.

Who owns the frontend auth tests? And whoever's on public-read, the contracts ordering is probably a one-line fix in the check script.



Send whichever suits. The first ends with a direct question about ownership, which is more likely to get someone to pick it up than a general flag.

That's Step 4. After this, the outstanding items are: your formatting PR waiting on review, #46 and #63 needing their acceptance criteria ticked, and the three remaining #27 follow-ups — ingestion performance, storage decision ownership, and documentation hygiene — none of which are urgent today.

is 46 and 63 done? if so we cna finish them?

Not quite, and the gaps are different for each.

#46 — everything substantive is delivered: tables, seed data, ERD. What's outstanding is process rather than work: the acceptance criteria aren't ticked, and the formatting PR isn't merged. Once that PR lands, tick the criteria and it can close.

#63 — one real gap remains. Its criterion is that expected statistics be documented independently of the derivation code. You have that in [evidence/validation/729307-published-figures.md](#), and 22 SQL assertions checking it. But the assertions live where? If they're in an ad-hoc script rather than the test suite, then nothing re-runs them, and the reference fixture doesn't protect against regression — which is the point of having one.

Check before deciding:

powershell

```
ls apps/backend/tests -Recurse -Filter *.ts | Select-Object -ExpandProperty FullName
```

Paste that. If the 22 assertions aren't in there, packaging them as a test is #63's remaining work — maybe an hour, and worth doing, because it's the thing that makes the reference fixture useful rather than decorative.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> ls apps/backend/tests -Recurse -
Filter *.ts | Select-Object -ExpandProperty FullName
C:\Users\BenSwartz\Sport-Analytics-Tool\apps\backend\tests\auth.test.ts
C:\Users\BenSwartz\Sport-Analytics-Tool\apps\backend\tests\health.test.ts
C:\Users\BenSwartz\Sport-Analytics-Tool\apps\backend\tests\test-app.ts
C:\Users\BenSwartz\Sport-Analytics-Tool\apps\backend\tests\api\public-
read.test.ts
C:\Users\BenSwartz\Sport-Analytics-Tool\apps\backend\tests\auth\token-
verifiers.ts
C:\Users\BenSwartz\Sport-Analytics-
Tool\apps\backend\tests\database\transaction.database.test.ts
C:\Users\BenSwartz\Sport-Analytics-Tool\apps\backend\tests\fixtures\domain-
builders.ts
C:\Users\BenSwartz\Sport-Analytics-Tool\apps\backend\tests\unit\database-
errors.test.ts
C:\Users\BenSwartz\Sport-Analytics-Tool\apps\backend\tests\unit\domain-
builders.test.ts
C:\Users\BenSwartz\Sport-Analytics-Tool\apps\backend\tests\unit\test-database-
safety.test.ts
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

No validation test. The 22 assertions aren't in the suite — they were run once by hand during #27 and nothing re-runs them. So #63's reference fixture currently proves the schema was right on 6 August, not that it stays right.

That's the remaining work, and it's worth doing: it converts a one-off verification into a regression test, and it's the thing that would catch a derivation bug when someone starts on aggregates.

Step 1 — find the assertions. They were run as SQL during #27; I need to see them before turning them into a test:

powershell

```
Get-ChildItem -Recurse -Include *.sql,*.md -Path apps\backend\scripts,database,evidenc
```

Paste the output. If they were never saved to a file — run ad hoc and only the result recorded in `729307-published-figures.md` — say so, and we'll write them fresh from that document instead.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> Get-ChildItem -Recurse -Include
*.sql,*.md -Path apps\backend\scripts,database,evidence,docs | Select-String -
Pattern "assert|expected|fall of wickets" -List | Select-Object -ExpandProperty
Path
C:\Users\BenSwartz\Sport-Analytics-
Tool\database\migrations\20260806150357535_delivery-event-schema.sql
C:\Users\BenSwartz\Sport-Analytics-Tool\database\seeds\README.md
C:\Users\BenSwartz\Sport-Analytics-Tool\evidence\ai\transcripts\dean-
feldman\2026-08-05-general-merge-issue-resolution.md
C:\Users\BenSwartz\Sport-Analytics-Tool\evidence\ai\transcripts\dean-
feldman\2026-08-05-issue-1517-azure-deployment-strategy.md
C:\Users\BenSwartz\Sport-Analytics-Tool\evidence\ai\transcripts\dean-
feldman\2026-08-06-general-event-data-submission-process.md
C:\Users\BenSwartz\Sport-Analytics-Tool\evidence\ai\transcripts\dean-
feldman\2026-08-06-general-sport-selection-for-analytics.md
C:\Users\BenSwartz\Sport-Analytics-Tool\evidence\ai\transcripts\dean-
feldman\2026-08-06-issue-72-testing-infrastructure-setup.md
C:\Users\BenSwartz\Sport-Analytics-Tool\evidence\ai\transcripts\dean-
feldman\2026-08-07-general-git-merge-conflict-resolution.md
C:\Users\BenSwartz\Sport-Analytics-Tool\evidence\ai\transcripts\dean-
feldman\2026-08-07-issue-72-testing-foundation-setup.md
C:\Users\BenSwartz\Sport-Analytics-Tool\evidence\ai\transcripts\gabriel-raz\AI
transcript Issue #14.md
C:\Users\BenSwartz\Sport-Analytics-Tool\evidence\ai\transcripts\shayna-
unterslak\2026-08-09-issue-47-public-read-api.md
C:\Users\BenSwartz\Sport-Analytics-Tool\evidence\sprints\sprint-1\2026-08-06-
planning.md
C:\Users\BenSwartz\Sport-Analytics-Tool\evidence\validation\729307-
published-figures.md
C:\Users\BenSwartz\Sport-Analytics-Tool\docs\architecture\system-
architecture.md
C:\Users\BenSwartz\Sport-Analytics-Tool\docs\data\cricsheet.md
C:\Users\BenSwartz\Sport-Analytics-Tool\docs\database\schema.md
C:\Users\BenSwartz\Sport-Analytics-Tool\docs\planning\project-backlog.md
C:\Users\BenSwartz\Sport-Analytics-Tool\docs\security\authentication.md
C:\Users\BenSwartz\Sport-Analytics-Tool\docs\git-methodology.md
C:\Users\BenSwartz\Sport-Analytics-Tool\docs\project_methodology.md
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

No SQL file holds them — they were run ad hoc. The expected figures survive in `evidence/validation/729307-published-figures.md`; the queries that checked them don't.

Show me that document so the test asserts the figures you actually verified rather than ones I reconstruct:

```
powershell
```

```
Get-Content evidence\validation\729307-published-figures.md
```

Paste it. Then I'll write the test against those numbers.

One decision while you fetch it: this test needs a seeded database, so it belongs with `transaction.database.test.ts` as a database test rather than a unit test — meaning it runs under `test:api` or whatever script covers database tests, and it will fail on any machine that hasn't seeded. That's acceptable, but the test should skip with a clear message rather than fail confusingly when the fixture is absent. I'll build that in.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> Get-Content evidence\validation\729307-published-figures.md # Published figures for validation "mat..."
```

PASTED

The assertions do exist — `apps/backend/scripts/validate-match.ts`. My earlier search was for `.sql` and `.md` files, so it missed a TypeScript script. It's what `db:validate` runs.

So #63's gap is narrower than I said: the assertions exist and are runnable, they're just not part of the test suite, so `npm run check` doesn't exercise them.

Before deciding whether that's worth changing, look at what's there:

```
powershell
```

```
Get-Content apps\backend\scripts\validate-match.ts -TotalCount 60
```

Paste that. If it's structured as a list of checks with expected values, wrapping it as a Vitest file is mechanical. If it's a script with console output and no return values, it's more work and arguably not worth it before Sprint 1.

Also worth noting two things that document itself flags as incomplete, both of which bear on whether #63 can close:

- The extras breakdown per type was derived from the database, not read from the scorecard, because the site blocks automated access. The document says it should be confirmed by hand before being treated as complete. Nobody has.
- The running score checkpoints and powerplay figures are recorded but not asserted.

Neither is a defect, but "expected statistics are documented independently of the derivation code" is only partly true while the extras split comes from the database it's meant to check.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> Get-Content  
apps\backend\scripts\validate-match.ts -TotalCount 60
```

```
/**
```

```
 * Derive innings figures from the database and compare them to the published  
 * scorecard recorded in evidence/validation/729307-published-figures.md.
```

```
 *
```

```
 * Every figure below is computed by SQL over delivery_current. Nothing is read  
 * from the source file. If the schema models extras, wickets or delivery  
 * ordering incorrectly, these numbers will disagree with the published ones.
```

```
 *
```

```
 * Usage:
```

```
 * npm run db:validate --workspace=@sport-analytics/backend -- <source-ref>
```

```
 */
```

```
import { readFileSync } from 'node:fs';
```

```
import { Client } from 'pg';
```

```
const sourceRef = process.argv[2] ?? '729307';
```

```
const client = new Client({
```

```
  connectionString: process.env.DATABASE_URL,
```

```
  ssl: {
```

```
    rejectUnauthorized: true,
```

```
    ca: readFileSync(new URL('../certs/supabase-ca.crt', import.meta.url), 'utf8'),
```

```
  },
```

```
});
```

```
let failures = 0;
```

```
function check(label: string, actual: unknown, expected: unknown): void {
```

```
  const pass = String(actual) === String(expected);
```

```
  if (!pass) failures += 1;
```

```
  console.log(
```

```
    `${pass ? 'PASS' : 'FAIL'} ${label}: ${actual}${pass ? '' :`
```

```
    (expected ${expected})`,
```

```
  );
```

```
}
```

```
async function main(): Promise<void> {
```

```
  await client.connect();
```

```
  console.log(Validating fixture ${sourceRef} against the published  
scorecard.\n`);
```

```
  // ---- innings totals -----
```

```
  // Team total is the sum of delivery totals PLUS innings-level penalty runs,
```

```
  // which belong to no delivery.
```

```
  const totals = await client.query(
```

```
    SELECT i.ordinal,
```

```
          t.name
```

```
      AS
```

```
team,
```

```
          SUM(d.runs_total)
```

```
          + COALESCE(i.penalty_pre, 0)
```

```
          + COALESCE(i.penalty_post, 0)
```

```
      AS
```

```
runs,
```

```
          COUNT(*) FILTER (WHERE d.extra_wides IS NULL) AS
```

```
legal_balls
```

```
    FROM innings i
```

```
    JOIN fixture f  ON f.fixture_id = i.fixture_id
```

```
    JOIN team t     ON t.team_id = i.batting_team_id
```

```
    JOIN delivery_current d ON d.innings_id = i.innings_id
```

```
    WHERE f.source_ref = $1
```

```
    GROUP BY i.ordinal, t.name, i.penalty_pre, i.penalty_post
```

```
ORDER BY i.ordinal],  
[sourceRef],  
);  
PS C:\Users\BenSwartz\Sport-Analytics-Tool>  
  
Show more
```

It's well structured — a `check(label, actual, expected)` helper with a failure counter. Converting to Vitest is mechanical: `check` becomes `expect(actual).toBe(expected)` inside `it` blocks, the connection moves to `beforeAll`, and the SQL stays as it is.

One thing in that excerpt is worth checking, though. The legal-ball count is:

```
sql  
  
COUNT(*) FILTER (WHERE d.extra_wides IS NULL) AS legal_balls
```

That counts no-balls as legal. A no-ball is not a legal delivery — it doesn't advance the over, same as a wide. The `ballNumber` fix we made today uses the correct rule, excluding both. So the validation script and the ingester now disagree about what a legal ball is.

It passed anyway because 729307's innings figures happen to work out, or because the assertion compares against a value that shares the error. Either way it's a latent bug in the thing that's meant to catch bugs, and it'll be wrong on any innings containing a no-ball where the count matters.

Show me the rest before I write anything:

```
powershell  
  
Get-Content apps\backend\scripts\validate-match.ts | Select-Object -Skip 60
```

Paste it. I want to see how `legal_balls` is asserted and what the other checks look like, then we can decide whether to fix the script in place or fix it while converting to a test.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> Get-Content apps\backend\scripts\validate-match.ts | Select-Object -Skip 60 // A no-ball is a legal deliver...
```

PASTED

I withdraw the concern about legal balls. There are two queries: the first one's `legal_balls` is computed but never used — the loop reads `overs.rows[index].legal_balls`, which excludes both wides and no-balls, correctly. The unused column in the totals query is dead weight, worth deleting, but not a bug.

The script is sound. What it lacks is being run by anything. Converting is mechanical.

Step 1 — create `apps/backend/tests/database/reference-fixture.database.test.ts`:

ts

```

/**
 * Validate the reference fixture against its published scorecard.
 *
 * Every figure is derived by SQL over delivery_current. Nothing is read from the
 * source file, so if the schema models extras, wickets or delivery ordering
 * incorrectly these numbers disagree with the published ones.
 *
 * Expected figures come from evidence/validation/729307-published-figures.md,
 * which records the scorecards they were taken from.
 *
 * The fixture must be present. Run `npm run db:seed` first.
 */

import { readFileSync } from 'node:fs';
import { resolve } from 'node:path';
import { Client } from 'pg';
import { beforeAll, afterAll, describe, expect, it } from 'vitest';

const SOURCE_REF = '729307';

const client = new Client({
  connectionString: process.env.DATABASE_URL,
  ssl: {
    rejectUnauthorized: true,
    ca: readFileSync(resolve(__dirname, '../../certs/supabase-ca.crt'), 'utf8'),
  },
});

beforeAll(async () => {
  await client.connect();
  const { rows } = await client.query(
    'SELECT count(*)::int AS n FROM fixture WHERE source_ref = $1',
    [SOURCE_REF],
  );
  if (rows[0].n === 0) {
    throw new Error(
      `Fixture ${SOURCE_REF} is not in the database. Run npm run db:seed before this t
    );
  }
});

afterAll(async () => {
  await client.end();
});

describe('reference fixture 729307', () => {
  const expected = [
    { ordinal: 0, team: 'Kings XI Punjab', runs: 132, wickets: 9, balls: 120, extras:

```

```
{ ordinal: 1, team: 'Kolkata Knight Riders', runs: 109, wickets: 10, balls: 110, e
];
```

```
it.each(expected)(
  'innings $ordinal totals agree with the published scorecard',
  async ({ ordinal, team, runs, wickets, balls, extras }) => {
    const { rows } = await client.query(
      `SELECT t.name AS team,
        SUM(d.runs_total)
          + COALESCE(i.penalty_pre, 0)
          + COALESCE(i.penalty_post, 0)                                AS runs,
        COUNT(*) FILTER (WHERE d.extra_wides IS NULL
                              AND d.extra_noballs IS NULL)           AS legal_balls,
        COALESCE(SUM(d.runs_extras), 0)                               AS extras,
        (SELECT COUNT(*)
          FROM delivery_current dc
          JOIN delivery_wicket w ON w.delivery_id = dc.delivery_id
          WHERE dc.innings_id = i.innings_id)                         AS wickets
      FROM innings i
      JOIN fixture f ON f.fixture_id = i.fixture_id
      JOIN team t ON t.team_id = i.batting_team_id
      JOIN delivery_current d ON d.innings_id = i.innings_id
      WHERE f.source_ref = $1 AND i.ordinal = $2
      GROUP BY i.innings_id, t.name, i.penalty_pre, i.penalty_post`,
      [SOURCE_REF, ordinal],
    );

    expect(rows).toHaveLength(1);
    expect(rows[0].team).toBe(team);
    expect(Number(rows[0].runs)).toBe(runs);
    expect(Number(rows[0].wickets)).toBe(wickets);
    expect(Number(rows[0].legal_balls)).toBe(balls);
    expect(Number(rows[0].extras)).toBe(extras);
  },
);

it('reproduces the complete fall of wickets', async () => {
  const { rows } = await client.query(
    `WITH d AS (
      SELECT dc.*,
        SUM(dc.runs_total) OVER (ORDER BY dc.innings_sequence) AS running_tota
        COUNT(*) FILTER (WHERE dc.extra_wides IS NULL
                              AND dc.extra_noballs IS NULL)
          OVER (ORDER BY dc.innings_sequence) AS legal_balls
      FROM delivery_current dc
      JOIN innings i ON i.innings_id = dc.innings_id
      JOIN fixture f ON f.fixture_id = i.fixture_id
      WHERE f.source_ref = $1 AND i.ordinal = 1
    )
```

```

SELECT d.running_total AS score,
       (d.legal_balls - 1) / 6 || '.' || ((d.legal_balls - 1) % 6 + 1) AS over,
       p.display_name AS player_out
FROM d
JOIN delivery_wicket w ON w.delivery_id = d.delivery_id
JOIN person p ON p.person_id = w.player_out_id
ORDER BY d.innings_sequence`,
[Source_REF],
);

expect(
  rows.map((r) => [Number(r.score), r.over, r.player_out]),
).toEqual([
  [13, '2.4', 'MK Pandey'],
  [19, '4.1', 'G Gambhir'],
  [19, '5.1', 'JH Kallis'],
  [50, '11.1', 'CA Lynn'],
  [59, '12.3', 'YK Pathan'],
  [62, '12.6', 'RV Uthappa'],
  [65, '13.4', 'PP Chawla'],
  [85, '15.6', 'SP Narine'],
  [103, '17.3', 'SA Yadav'],
  [109, '18.2', 'UT Yadav'],
]);

});

it('excludes dismissals not credited to the bowler', async () => {
  const { rows } = await client.query(
    `SELECT COUNT(*) FILTER (WHERE k.credits_bowler)      AS credited,
       COUNT(*) FILTER (WHERE NOT k.credits_bowler) AS uncredited
       FROM delivery_current d
       JOIN innings i ON i.innings_id = d.innings_id
       JOIN fixture f ON f.fixture_id = i.fixture_id
       JOIN delivery_wicket w ON w.delivery_id = d.delivery_id
       JOIN dismissal_kind k ON k.code = w.kind
       WHERE f.source_ref = $1`,
    [Source_REF],
  );

  expect(Number(rows[0].credited)).toBe(17);
  expect(Number(rows[0].uncredited)).toBe(2);
});

it('credits Sandeep Sharma with three wickets, matching the player of the match line
  const { rows } = await client.query(
    `SELECT COUNT(*) AS wickets
       FROM delivery_current d
       JOIN innings i ON i.innings_id = d.innings_id
       JOIN fixture f ON f.fixture_id = i.fixture_id

```

```

    JOIN delivery_wicket w ON w.delivery_id = d.delivery_id
    JOIN dismissal_kind k ON k.code = w.kind
    JOIN person p ON p.person_id = d.bowler_id
    WHERE f.source_ref = $1 AND k.credits_bowler AND p.display_name = 'Sandeep Sha
[SOURCE_REF],
);

expect(Number(rows[0].wickets)).toBe(3);
});
});

```

Two guesses I can't verify: 'Sandeep Sharma' as the display name — Cricsheet may render it differently — and the 17/2 credited split, which I've taken from the evidence document's prose. If either is wrong the test will say so.

Step 2 — run it:

```

powershell

npx vitest run tests/database/reference-fixture.database.test.ts --root apps/backend

```

Paste the output.

```

PS C:\Users\BenSwartz\Sport-Analytics-Tool> npx vitest run
tests/database/reference-fixture.database.test.ts --root apps/backend
RUN v2.1.9 C:/Users/BenSwartz/Sport-Analytics-Tool/apps/backend
> tests/database/reference-fixture.database.test.ts (5)
| [ beforeAll ]
↓ reference fixture 729307 (5) [skipped]
  ↓ innings +0 totals agree with the published scorecard [skipped]
  ↓ innings 1 totals agree with the published scorecard [skipped]
  ↓ reproduces the complete fall of wickets [skipped]
  ↓ excludes dismissals not credited to the bowler [skipped]
  ↓ credits Sandeep Sharma with three wickets, matching the player of the match
line [skipped]
_____ Failed Suites 1
_____
FAIL tests/database/reference-fixture.database.test.ts [
tests/database/reference-fixture.database.test.ts ]
{
  stack: 'AggregateError: \n' +
    '   at internalConnectMultiple (node:net:1193:18)\n' +
    '   at afterConnectMultiple (node:net:1783:7)',
  errors: [
    {
      stack: 'Error: connect ECONNREFUSED ::1:5432\n' +
        '   at createConnectionError (node:net:1746:14)\n' +
        '   at afterConnectMultiple (node:net:1776:16)',
      message: 'connect ECONNREFUSED ::1:5432',
      errno: -4078,
    }
  ]
}

```

```

code: 'ECONNREFUSED',
syscall: 'connect',
address: '::1',
port: 5432,
constructor: 'Function<Error>',
name: 'Error',
toString: 'Function<toString>'
},
{
  stack: 'Error: connect ECONNREFUSED 127.0.0.1:5432\n' +
    '   at createConnectionError (node:net:1746:14)\n' +
    '   at afterConnectMultiple (node:net:1776:16)',
  message: 'connect ECONNREFUSED 127.0.0.1:5432',
  errno: -4078,
  code: 'ECONNREFUSED',
  syscall: 'connect',
  address: '127.0.0.1',
  port: 5432,
  constructor: 'Function<Error>',
  name: 'Error',
  toString: 'Function<toString>'
}
],
code: 'ECONNREFUSED',
stackStr: 'AggregateError: \n' +
  '   at internalConnectMultiple (node:net:1193:18)\n' +
  '   at afterConnectMultiple (node:net:1783:7)',
nameStr: 'AggregateError',
expected: 'undefined',
actual: 'undefined',
message: '',
constructor: 'Function<AggregateError>',
name: 'AggregateError',
toString: 'Function<toString>',
stacks: []
}

```

—[1/1]—

Test Files 1 failed (1)

Tests 5 skipped (5)

Start at 14:48:16

Duration 1.19s (transform 451ms, setup 0ms, collect 553ms, tests 62ms,
environment 0ms, prepare 202ms)

PS C:\Users\BenSwartz\Sport-Analytics-Tool>

Show more

It tried localhost:5432 — `DATABASE_URL` wasn't loaded. `npm run vitest` directly doesn't pick up `.env`; the other scripts pass `--env-file=.env`.

Check how the existing database test does it, since there's already one that connects:

powershell

```
Get-Content apps\backend\tests\database\transaction.database.test.ts -TotalCount 40
```

powershell

```
Select-String -Path apps\backend\package.json -Pattern "test"
```

Paste both. There's probably a `test:database` script with the right env handling, and a vitest config with a setup file — my test should use whatever connection helper that one uses rather than opening its own client.

Also worth noting: `transaction.database.test.ts` exists and presumably connects to a test database, not the shared development one. If so, my test is wrong to expect the seed fixture in it, and the right shape is to seed the test database as part of the test setup. Let's see what's there.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> Get-Content
apps\backend\tests\database\transaction.database.test.ts -TotalCount 40
import { Pool } from 'pg';
import { afterAll, beforeAll, describe, expect, test } from 'vitest';
import { executeQuery, withTransaction } from '../src/database';
import { assertSafeTestDatabase } from '../scripts/test-database-safety';
describe('database transaction helper', () => {
  let pool: Pool | undefined;
  function getPool(): Pool {
    if (!pool) {
      throw new Error('Test database pool has not been initialised.');
```

Show more

```
    }
    return pool;
  }
  beforeAll(() => {
    const databaseUrl = assertSafeTestDatabase(
      process.env.DATABASE_URL_TEST,
      process.env.DATABASE_URL,
      process.env.NODE_ENV,
    );
    pool = new Pool({
      connectionString: databaseUrl.toString(),
    });
  });
  afterAll(async () => {
    if (pool) {
      await pool.end();
    }
  });
  test('commits a successful multi-step transaction', async () => {
    const databasePool = getPool();
    const originalResult = await executeQuery<{ note: string }>(
      databasePool,
    );
  });
});
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

That changes the design. There's a deliberate safety mechanism: `assertSafeTestDatabase` requires `DATABASE_URL_TEST` and refuses to run destructive tests against the development database. Dean built it as part of #72.

So my test is wrong in shape. It connects to whatever `DATABASE_URL` points at — the shared Supabase database — which bypasses the guard, and it depends on the dev seed being present.

The correct shape: connect to the test database via the same guard, seed the reference fixture into it as part of setup, then assert. That makes the test self-contained and safe.

Before I rewrite it, I need to know what exists:

```
powershell
```

```
Get-Content apps\backend\scripts\test-database-safety.ts
```

```
powershell
```

```
Select-String -Path apps\backend\package.json -Pattern "test|DATABASE_URL_TEST"
```

Paste both. Specifically: is there a test database configured at all, does it get migrated, and which npm script runs the `.database.test.ts` files.

Also worth saying — this is now more than "an hour" of work. If you'd rather stop here, ticking #63's delivered criteria and leaving the test as its remaining item is perfectly defensible; `db:validate` exists and runs, it just isn't automated.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> Get-Content
apps\backend\scripts\test-database-safety.ts
export function assertSafeTestDatabase(
  testDatabaseUrl: string | undefined,
  developmentDatabaseUrl: string | undefined,
  nodeEnvironment: string | undefined,
): URL {
  if (nodeEnvironment !== 'test') {
    throw new Error('Test database commands require NODE_ENV=test.');
```

```
  }
  if (!testDatabaseUrl) {
    throw new Error('DATABASE_URL_TEST is required.');
```

```
  }
  if (developmentDatabaseUrl && testDatabaseUrl === developmentDatabaseUrl)
  {
    throw new Error('DATABASE_URL_TEST must not match DATABASE_URL.');
```

```
  }
  let parsedUrl: URL;
  try {
    parsedUrl = new URL(testDatabaseUrl);
  } catch {
    throw new Error('DATABASE_URL_TEST must be a valid URL.');
```

```
  }
}
```

```
if (parsedUrl.protocol !== 'postgres:' && parsedUrl.protocol !== 'postgresql:') {  
  throw new Error('DATABASE_URL_TEST must use the PostgreSQL protocol.');
```

```
}  
const databaseName = parsedUrl.pathname.replace(/^\//, "").toLowerCase();  
if (!databaseName.includes('test')) {  
  throw new Error('The test database name must contain "test".');
```

```
}  
return parsedUrl;  
}  
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

Show more

That's a well-built guard — requires `NODE_ENV=test`, a distinct URL, and "test" in the database name. Four independent checks before anything touches a database.

The problem for you: **there is no test database**. Your project's Postgres is one hosted Supabase instance, and the free plan gives you one database. Creating a second one named something containing "test" is possible on the same instance, but it doubles the storage against a 500 MB limit that ADR-003 already records as unresolved.

So the honest position is that a database-backed reference test can't be written properly today. It's blocked on the storage decision — the same one that's been sitting unowned since 5 August.

I'd stop here on #63. Specifically:

Delete the test file I had you create. It bypasses the safety guard and points at the shared database, and leaving it around invites someone to run it.

powershell

```
Remove-Item apps\backend\tests\database\reference-fixture.database.test.ts
```

Comment on #63 recording what's done, what isn't, and why:

markdown

Seed data and the invalid submission examples are delivered by the pull request linked above. Expected figures for the reference fixture are documented in ``evidence/validation/729307-published-figures.md`` and asserted by ``apps/backend/scripts/validate-match.ts``, which derives every figure by SQL over ``delivery_current`` and reads nothing from the source file.

The remaining criterion is that those assertions run as part of the automated suite rather than on demand. That is currently blocked: ``assertSafeTestDatabase`` correctly requires a separate database whose name contains "test", and no such database exists. Creating one on the current instance would double storage against a limit ADR-003 already records as unresolved.

This should be revisited once the storage decision in ADR-003 consequence 4 is taken.

AI Declaration: The preceding comment was generated with the assistance of Claude-Web[

The storage decision is now blocking three things — the measured benchmark, any realistic dataset, and this test. It has no owner and no date, and it's five days old. That's Tuesday's most important item.

can't I do the storage thing then?

You can do the analysis. You can't take the decision alone — it's a group commitment to either pay money, cut agreed scope, or migrate provider, and ADR-003 says the team must decide.

But the reason it's sat five days is probably that nobody has laid out the options with numbers. That part is yours, and it's the useful contribution: turn "we might exceed the limit" into "here are three options, here's what each costs, here's what I recommend," and Tuesday becomes a fifteen-minute decision instead of another deferral.

What you'd need:

- **A real storage measurement.** The current figure is 128 bytes per delivery of heap from four fixtures, with indexes at their floor. Blocked on ingestion being 68 seconds a fixture — which is the thing to fix first, and which is separately worth fixing.
- **What Supabase actually charges** above the free plan, and whether Wits or the course provides anything.
- **What the alternatives are.** Object storage for source files and dataset releases is named in your handover as untested. Azure Postgres is already in your deployment plan. Reducing scope from eleven competitions to two or three is the cheapest option and the one nobody wants to say out loud.

My honest read: **fix ingestion performance first**. It unblocks the measurement, which is the one input the decision genuinely lacks, and it's a real Sprint 2 blocker regardless of what the storage answer turns out to be. Two hours of work that makes the rest tractable.

Want to start there?

wait, first, can we close 43?

No — #43 is barely started, and it's the one I'd leave well alone.

What exists is `app_user` with `auth_provider`, `auth_subject` and `application_role`.

What #43 asks for that doesn't exist: approved-submitter status, submission scope tied to the competition model, the constraints enforcing both, and a migration creating them.

More to the point, it's blocked on a decision nobody has made: **what a submission scope actually is**. A competition? A season? A set of fixtures? That choice determines the schema, and it constrains #44, #45 and #51 — all three of which are unstarted and all three of which depend on it.

So #43 isn't a closeable issue, it's a design question wearing an issue's clothes. Worth raising Tuesday alongside storage: two blocking decisions, both undecided, both holding up multiple issues.

What you *can* do now is tick its one satisfied criterion and leave the comment we drafted earlier, so the partial progress is visible.

Then — ingestion performance?

okay cool

Ingestion performance. Step 1 — find out where the time actually goes.

68 seconds for roughly 240 deliveries is about 280ms per delivery, which is far too slow to be anything but round trips. But `ingest-match.ts` issues several queries per delivery — the insert, then wickets, fielders, reviews, replacements — so the count matters before optimising.

Time a single ingest with the query count visible. Add a counter to the file temporarily, or simpler: reason about it first.

Per delivery, minimum one `INSERT INTO delivery`. Plus per wicket a `delivery_wicket` insert and a `scalar` call. Plus fielders, reviews, replacements. Call it 1.2 queries per delivery average, so ~290 round trips for 240 deliveries — which would mean roughly 230ms per round trip.

That's high even for a pooler to Frankfurt from Johannesburg. Measure it rather than guess:

```
powershell
```

```
Measure-Command { npm run db:sql --workspace=@sport-analytics/backend -- scripts/queri
```

That's one connection plus one query, so it gives you the fixed overhead. Then I want a round-trip figure — create `apps/backend/scripts/queries/ping.sql`:

```
sql
```

```
SELECT 1;
```

and run it the same way. The difference between the two tells you connection setup versus query time.

Paste both numbers.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> Measure-Command { npm run
db:sql --workspace=@sport-analytics/backend -- scripts/queries/state.sql } |
Select-Object TotalSeconds
TotalSeconds
-----
3.2989049
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

3.3 seconds for the whole process — but most of that is npm and tsx starting up, not the database. Need to separate them.

Create `apps/backend/scripts/latency.ts`:

```
ts
```

```

/**
 * Measure round-trip latency to the database, to distinguish connection setup
 * from per-query cost.
 */

import { readFileSync } from 'node:fs';
import { Client } from 'pg';

const client = new Client({
  connectionString: process.env.DATABASE_URL,
  ssl: {
    rejectUnauthorized: true,
    ca: readFileSync(new URL('../certs/supabase-ca.crt', import.meta.url), 'utf8'),
  },
});

async function main(): Promise<void> {
  const connectStart = Date.now();
  await client.connect();
  console.log(`connect: ${Date.now() - connectStart} ms`);

  const samples: number[] = [];
  for (let i = 0; i < 20; i += 1) {
    const start = Date.now();
    await client.query('SELECT 1');
    samples.push(Date.now() - start);
  }

  samples.sort((a, b) => a - b);
  const total = samples.reduce((a, b) => a + b, 0);
  console.log(`queries: ${samples.length}`);
  console.log(`  min    : ${samples[0]} ms`);
  console.log(` median : ${samples[Math.floor(samples.length / 2)]} ms`);
  console.log(`  max    : ${samples[samples.length - 1]} ms`);
  console.log(`  mean   : ${(total / samples.length).toFixed(1)} ms`);
}

main()
  .catch((error: unknown) => {
    console.error(error instanceof Error ? error.message : error);
    process.exitCode = 1;
  })
  .finally(() => {
    void client.end();
  });

```

Add to `apps/backend/package.json`:

```
json
```

```
"db:latency": "tsx --env-file=.env scripts/latency.ts"
```

Run it:

```
powershell
```

```
npm run db:latency --workspace=@sport-analytics/backend
```

Paste the output. If the median round trip is ~200ms, the fix is batching — fewer, bigger statements. If it's ~20ms, the time is going somewhere else and I'm wrong about the cause.

```
PS C:\Users\BenSwartz\Sport-Analytics-Tool> npm run db:latency --
workspace=@sport-analytics/backend
> @sport-analytics/backend@0.1.0 db:latency
> tsx --env-file=.env scripts/latency.ts
connect: 1233 ms
queries: 20
  min   : 173 ms
  median : 181 ms
  max   : 248 ms
  mean  : 185.8 ms
PS C:\Users\BenSwartz\Sport-Analytics-Tool>
```

That's the answer, and it's not a code problem.

185ms median round trip. Johannesburg to Frankfurt is roughly 160ms of physics — you cannot optimise your way past the speed of light. The ingester's 68 seconds is ~290 round trips at 185ms plus overhead, which accounts for essentially all of it.

Three consequences worth being precise about:

Batching is the fix, and it's a large win. Collapsing per-delivery inserts into one multi-row `INSERT ... VALUES` per innings takes ~290 round trips down to under 20. Sixty-eight seconds becomes about four. That's real and worth doing.

But this also means every API request pays 185ms before any query work happens. Your ADR-003 already notes the latency risk with Azure South Africa North as the app host. The intermediate tier requires a stated response time under load — with a 185ms floor per query, an endpoint doing three sequential queries starts at 555ms before it does anything useful. That's a bigger finding than the ingestion rate, and it belongs in front of the group rather than in a script.

It strengthens the storage conversation. "Move provider" now has a second reason behind it — not just the 500 MB limit but the region. Azure Postgres in South Africa North would be single-digit milliseconds from an app in the same region.

So: batch the ingester, and raise the latency finding. Which first?

you decide

Raise the latency finding first — it's ten minutes, it changes what other people are building right now, and the batching work will still be there afterwards.

The reason for the order: Shayna and whoever's on `feat/47-public-read-api` are writing endpoints today against a database with a 185ms floor. If an endpoint does three sequential queries it starts at 555ms before doing anything useful, and that shapes how they write it — batched queries, fewer round trips, caching. Better they know now than after the code exists.

Two things to produce:

A Gitea issue recording the measurement, so it's evidence rather than chat.

A WhatsApp message so people see it today.

I'd also add the measurement to ADR-003 as a second consequence — the ADR currently notes latency as a risk in the abstract, and you now have a number. But that's a follow-up, not today.

Want the issue text first?

yes

Title:

Database round-trip latency is 185 ms and constrains API response times

Type: Task. **Body:**

markdown

Description

Every database query from a developer machine in Johannesburg to the hosted Supabase instance costs approximately 185 ms of round-trip latency. This is a property of the distance between the client and the database region, not of the queries.

Measured on 10 August 2026 with `apps/backend/scripts/latency.ts`, twenty sequential `SELECT 1` statements over an established connection:

| Measure | Value |
|------------------|----------|
| Connection setup | 1233 ms |
| Minimum | 173 ms |
| Median | 181 ms |
| Mean | 185.8 ms |
| Maximum | 248 ms |

Motivation

The intermediate tier requires the platform to meet a stated response time under load. With a 185 ms floor per query, an endpoint issuing three sequential queries starts at 555 ms before any query work is done. No amount of indexing or schema design reduces this; the only levers are issuing fewer queries per request, serving repeated reads from cache, and the distance between the application and the database.

The same floor explains the ingestion rate. Loading one fixture takes approximately 68 seconds, which is roughly 290 round trips at 185 ms. That is a separate issue, but it has the same cause.

ADR-003 already records latency as a consequence of the hosted database decision, on the basis that the application will be deployed to Azure South Africa North while the database is in `eu-central-1`. This issue supplies the measurement that consequence lacked.

Acceptance Criteria

- [] The measurement is recorded in ADR-003 alongside the consequence it quantifies
- [] Endpoints are written to issue as few sequential queries per request as the work allows, rather than one query per entity
- [] A target response time is stated and tested against, taking the latency floor into account
- [] The team decides whether the latency, together with the storage limit, justifies revisiting the database host

Dependencies

Relates to the storage decision in ADR-003 consequence 4. Both point at the same question: whether the current hosted instance is the right place for this platform. Neither has an owner.

Evidence

`apps/backend/scripts/latency.ts` and the figures above. The script is committed so the measurement can be repeated from any member's machine, which will also show whether the figure varies by location.

AI Declaration: The preceding issue was generated with the assistance of Claude-Web[CL

Labels: `area: database`, `area: api`, `priority: high`. Milestone Sprint 1 — the decision is Sprint 1 even if the remedies aren't.

Note the last line of Evidence: worth asking a teammate to run `db:latency` too. If someone on a different connection sees 40ms, my diagnosis is wrong and it's a local network problem, not geography.

done

Commit the script so others can run it — you're on `main` with an uncommitted `latency.ts`, and your methodology forbids committing there.

```
powershell
git switch -c chore/NN-measure-database-latency
```

Replace `NN` with the issue number you just got. Then:

```
powershell
git add apps/backend/scripts/latency.ts apps/backend/package.json
git commit -m "chore(db): add a database round-trip latency measurement" -m "Measures
Median round trip from Johannesburg to the eu-central-1 instance is 181 ms.

Refs #NN
Assisted-by: Claude-Web[Claude Opus 5]"
git push -u origin chore/NN-measure-database-latency
```

Tell me the issue number and I'll give you the PR description and the WhatsApp message.

powershell

```
git switch -c chore/105-measure-database-latency
git add apps/backend/scripts/latency.ts apps/backend/package.json
git commit -m "chore(db): add a database round-trip latency measurement" -m "Measures"
```

Median round trip from Johannesburg to the eu-central-1 instance is 181 ms.

Refs #105

Assisted-by: Claude-Web[Claude Opus 5]"

```
git push -u origin chore/105-measure-database-latency
```

PR title: #105 Add a database round-trip latency measurement

markdown

Purpose

Supplies the measurement behind #105. ADR-003 records latency as a consequence of the hosted database decision but gives no figure; this makes it measurable and repeatable from any member's machine.

Changes

- `apps/backend/scripts/latency.ts` and the `db:latency` script. It times connection setup once, then twenty sequential `SELECT 1` statements over the established connection, reporting min, median, mean and max.

Testing

Run from Johannesburg on 10 August 2026: connection 1233 ms, median round trip 181 ms, mean 185.8 ms over twenty samples.

Known Limitations

One machine, one location, one moment. Running it from another member's machine would show whether the figure is geographic or local. If someone on a different connection sees a materially lower number, the conclusion in #105 needs revising.

Related Issue

Refs #105

Now the message.

A Full finding + ask B Shorter

Heads up for anyone writing endpoints — every DB query from here is ~185ms round trip. Measured it, median 181ms over 20 samples (script's in #105, npm run db:latency).

That's distance to the database region, not slow queries — indexing won't help. An endpoint doing 3 sequential queries starts at 555ms before doing anything useful. Worth designing for fewer round trips per request.

Same reason ingestion is 68s a match.

Could someone else run it and compare? If you get 40ms it's my connection and I'm wrong.



Send the first — the request for a second measurement is the part that matters, and it's what stops this being an assertion.

Once that's out, batching the ingester is the next piece of work, and it's now well-motivated: ~290 round trips down to under 20, 68 seconds down to about four.

next?

Batch the ingester. The plan, before any code:

The deliveries loop currently issues one `INSERT` per delivery and reads back `delivery_id` to attach wickets, reviews and replacements. Batching means collecting an innings' deliveries into a single multi-row `INSERT ... VALUES (...), (...), ...` with `RETURNING delivery_id`, then using the returned ids for the dependent rows — which themselves batch the same way.

Two things that make this trickier than it looks, and worth deciding before writing:

ON CONFLICT DO NOTHING breaks the id mapping. Currently a skipped row returns nothing and the code `continue`s. In a batch, `RETURNING` gives you back fewer rows than you sent, and you can't tell which. Fix: also return the natural key columns, and map results by `(over_number, position_in_over)` rather than by array position.

Wickets need their delivery id. So it's two passes per innings: insert all deliveries, build a map from natural key to id, then batch the wickets, reviews and replacements against that map.

Expected result: from ~290 round trips to roughly 6 per innings — deliveries, wickets, fielders, reviews, replacements, plus the innings row itself.

Before I write it, one question I can't answer from here: **is this worth doing to `ingest-match.ts` at all?** Its own header says it "is not the ingestion pipeline, which is a separate concern" and "makes no attempt to be fast." Optimising a script that explicitly disclaims performance may be the wrong target — the batch pipeline is Sprint 2 work and would be a new module.

The case for doing it anyway: the seed uses it, and four minutes versus twenty seconds matters to everyone who seeds.

Your call — batch this script, or leave it and let Sprint 2's pipeline be where batching lands?

i just want to fix work, not add more, you decide

Then leave it. `ingest-match.ts` says outright it makes no attempt to be fast, the seed takes four minutes once, and batching it is an hour of fiddly work on a script that Sprint 2 will replace. It's optimising the wrong thing.

What actually needs finishing, in order:

- 1. Your formatting PR** — still open, waiting on review. Chase it.
- 2. Tick and close #46.** Everything substantive is delivered. Once the formatting PR merges, tick the criteria and close it.
- 3. Comment on #63 and leave it open** — the wording we drafted, recording that the reference test is blocked on there being no test database.
- 4. Comment on #43** and tick its one criterion. Also open.
- 5. Your AI register** — you've done a fair bit today that isn't recorded: #84, the seed work, the ERD, the latency measurement. That's a 10 August row.

That's the lot. Nothing new, all of it closing out work that's already done.

Start with 5 — it's the one that's bitten you twice today. Want the row?

yes give me the row

```
powershell
```

```
@'
```

```
"2026-08-10","Ben Swartz","Claude Web","Claude Opus 5","Planning; technical guidance;  
'@ | Add-Content -Path evidence\ai\registers\ben-swartz.csv -Encoding utf8
```

Check it landed on its own line:

powershell

```
Get-Content evidence\ai\registers\ben-swartz.csv | Select-Object -Last 2
```

Then export today's session as `2026-08-10-continuous-project-session.pdf` into `evidence\ai\transcripts\ben-swartz\`.

You're on `chore/105-measure-database-latency`, so it can go in that branch rather than needing its own:

powershell

```
git add evidence/ai/registers/ben-swartz.csv evidence/ai/transcripts/ben-swartz/  
git commit -m "docs(evidence): register AI usage for 10 August" -m "Refs #105  
Assisted-by: Claude-Web[Claude Opus 5]"  
git push
```



     2 minutes ago

